PRESIDENCY UNIVERSITY

Private University Estd. in Karnataka State by Act No. 41 of 2013

Itgalpura, Rajankunte, Yelahanka, Bengaluru – 560064

HOSPITAL MANAGEMENT SYSTEM

AN INTERNSHIP REPORT

Submitted by

RUCHITHA .M- 20221CSE0708

Under the guidance of,

Mr. Jerrin Joe Francis

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

GAIN MORE KNOWLEDGE
REACH GREATER HEIGHTS

PRESIDENCY UNIVERSITY

BENGALURU

APRIL 202

PRESIDENCY UNIVERSITY

Private University Estd. in Karnataka State by Act No. 41 of 2013

Itgalpura, Rajankunte, Yelahanka, Bengaluru – 560064

PRESIDENCY SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

BONAFIDE CERTIFICATE

Certified that this report **"Hospital Management System"** is a bonafide work of **RUCHITHA.M (20221CSE0708),** who has successfully carried out the internship work and submitted the report for partial fulfillment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE AND ENGINEERING during 2025-2026.

Mr. Jerrin Joe Francis Internship Guide PSCS Presidency University	**Dr. Balakrishnan . R** Program Internship Coordinator PSCS Presidency University	**Dr. Md Ziaur Rahman** School Internship Coordinator PSCS Presidency University
Dr. Asif Mohammed Head of the Department, PSCS Presidency University	**Dr. Duraipandian N** Dean PSCS / PSIS Presidency University	

Name and Signature of the Examiners

Sl.No	Name of the Examiner	Signature/Date
1	Mr. H Senthil Kumar	27/04/26
2	Ms. Attar Mahay Sheetal	27/04

PRESIDENCY UNIVERSITY

PRESIDENCY SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

DECLARATION

I am a student of final year B.Tech in COMPUTER SCIENCE AND ENGINEERING, at Presidency University, Bengaluru, named Ruchitha M, hereby declare that the internship work titled **"Hospital Management System"** has been independently carried out by me and submitted in partial fulfillment for the award of the degree of B.Tech in COMPUTER SCIENCE AND ENGINEERING during the academic year of 2025-2026. Further, the matter embodied in the internship has not been submitted previously by anybody for the award of any Degree or Diploma to any other institution.

Name of the Student: RUCHITHA .M USN: 20221CSE0708 Signature:

PLACE: BENGALURU

DATE:23-April2026

INTERNSHIP COMPLETION CERTIFICATE

रेल पहिया कारखाना
RAIL WHEEL FACTORY
भारत सरकार / Govt. of India
रेल मंत्रालय / Ministry of Railways

महा प्रबंधक का कार्यालय,
कार्मिक विभाग
यलहंका, बैंगलूरु - 560 064
General Manager's Office,
Personnel Department
Yelahanka, Bangalore - 560 064

NO.RWF/NG-16/348/Pt III

DATE: 22.04.2026

CERTIFICATE

This is to certify that **Ms. RUCHITHA M, (20221CSE0708)**, B.Tech (Computer Science & Engineering) Student of Presidency University, has underwent Internship on **"Full Stack Web Development-Hospital Management System" at Rail Wheel Factory, Yelahanka, Bangalore** for a period of 2.5 months i.e., 02nd February 2026 to 16th April 2026.

The student has shown keen interest in learning during the period she was with us for the Internship. We found her Enthusiastic, Diligent, Hard Working and Creative.

We wish a bright future and success in all her endeavors.

(K.S.KIRAN)

SENIOR PERSONNEL OFFICER-I

Tel : +91-80-28072402 Fax : +91-80-28460306

ACKNOWLEDGEMENTS

For completing this internship work, I have received the support and the guidance from many people whom I would like to mention with deep sense of gratitude and indebtedness. I extend my gratitude to our beloved Chancellor, Pro-Vice Chancellor, and Registrar for their support and encouragement in completion of the internship.

I would like to sincerely thank my internal guide **Mr. Jerrin Joe Francis**, Presidency School of Computer Science and Engineering, Presidency University, for his/her moral support, motivation, timely guidance and encouragement provided to me during the period of internship work.

I am also thankful to **Dr. Asif Mohamed H B**, Professor, Head of the Department, Presidency School of Computer Science and Engineering Presidency University, for his/her mentorship and encouragement.

I express my cordial thanks to **Dr. Duraipandian N**, Dean PSCS & PSIS and the Management of Presidency University for providing the required facilities and intellectually stimulating environment that aided in the completion of my internship work.

I am grateful to **Dr. Md Ziaur Rahaman,** Internship Coordinator and **Dr. Balakrishnam,** Program Internship Coordinator, Presidency School of Computer Science and Engineering, or facilitating problem statements, coordinating reviews, monitoring progress, and providing their valuable support and guidance.

I am also grateful to Teaching and Non-Teaching staff of Presidency School of Computer Science and Engineering and also staff from other departments who have extended their valuable help and cooperation.

RUCHITHA .M

ABSTRACT

The rapid growth of digital technologies has significantly transformed the healthcare sector by improving the efficiency and accuracy of hospital operations. Traditional hospital management methods rely heavily on manual record keeping, which often leads to data redundancy, delays in accessing patient information, and increased chances of human error. With the increasing number of patients and the need for organized healthcare services, there is a strong demand for a reliable and automated system that can manage hospital activities effectively.

The Admin has full control over the system, including managing users and viewing patient records. The Receptionist is responsible for adding new patients and viewing patient details, while the Doctor can access patient records and update diagnosis and treatment information. The application follows a client-server architecture where the frontend interacts with backend APIs to perform operations such as login authentication, patient management, and data retrieval. The system ensures structured data handling, role-based access control, and efficient communication between different modules.

TABLE OF CONTENTS

Sl. No.	Title	Page No.
	Declaration	iii
	Acknowledgement	v
	Abstract	vi
	List of Figures	ix
	List of Tables	x
	Abbreviations	xi
1.	Introduction 1.1 Background 1.2 Statistics of Internship 1.3 Prior existing technologies 1.4 Proposed approach 1.5 Objectives 1.6 SDGs 1.7 Overview of internship report	1
2.	Literature Review	17
3.	Methodology	20
4.	Project Management 4.1 Internship timeline 4.2 Risk analysis 4.3 Internship budget	30
5.	Analysis and Design 5.1 Requirements 5.2 Block Diagram 5.3 System Flow Chart 5.4 Choosing devices 5.5 Designing units 5.6 Standards 5.7 Mapping with IoTWF reference model layers 5.8 Domain model specification	36

	5.9 Communication model 5.10 IoT deployment level 5.11 Functional view 5.12 Mapping IoT deployment level with functional view 5.13 Operational view 5.14 Other Design	
6.	Hardware, Software and Simulation 6.1 Hardware 6.2 Software development tools 6.3 Software code 6.4 Simulation	54
7.	Evaluation and Results 7.1 Test points 7.2 Test plan 7.3 Test result 7.4 Insights	58
8.	Social, Legal, Ethical, Sustainability and Safety Aspects 8.1 Social aspects 8.2 Legal aspects 8.3 Ethical aspects 8.4 Sustainability aspects 8.5 Safety aspects	65
9.	Conclusion	71
	References	75
	Base Paper (Optional)	77
	Appendix	78

LIST OF FIGURES

Sl. No.	Figure Name	Caption	Page No.
1	Fig 1.1	Sustainable Development Goals	12
2	Fig 3.1	The V Model methodology	21
3	Fig 3.2	The Devops methodology	19
4	Fig 4.1	Gantt Chart Illustrating the Project Schedule and Progress	22
5	Fig 5.1	Block Diagram	28
6	Fig 5.2	System Flow Chart	30
7	Fig 5.3	UML Class Diagram Representation	33
8	Fig 5.4	Communication Model	35
9	Fig 5.5	Operational view	40

LIST OF TABLES

Sl. No.	Table Name	Table Caption	Page No.
1	Table 2.1	Summary of Literature Review	17
2	Table 3.1	Phases of DevOps Model	20
3	Table 4.1	Project Implementation Timeline	23
4	Table 4.2	PESTLE – Based Risk Analysis	24
5	Table 4.3	Internship Project Budget	26
6	Table 5.1	Summarizing Requirements	35
7	Table 5.2	Standards Adopted Across Domains	41
8	Table 5.3	Description of Domain model	42

ABBREVIATIONS

Abbreviation	Full Form
URL	Uniform Resource Locator
VPN	Virtual Private Network
XML	Extensible Markup Language
API	Application Programming Interface
HTTP	HyperText Transfer Protocol
JSON	JavaScript Object Notation
URL	Uniform Resource Locator
VPN	Virtual Private Network
XML	Extensible Markup Language
DBMS	Database Management System
SQL	Structured Query Language
CRUD	Create, Read, Update, Delete
REST	Representational State Transfer
SDLC	Software Development Life Cycle
DevOps	Development and Operations
CPU	Central Processing Unit
GDPR	General Data Protection Regulation
DPDPA	Digital Personal Data Protection Act

Chapter1

INTRODUCTION

The healthcare sector has increasingly adopted digital solutions to improve the efficiency and accuracy of hospital operations. Traditional methods of managing patient records and hospital activities are mostly manual, which often leads to errors, duplication of data, and delays in accessing important information. As hospitals handle a large number of patients daily, maintaining organized records and ensuring smooth communication between staff members becomes a challenging task. This creates a need for an automated system that can streamline hospital processes and improve overall service quality.

The main aim of this project is to develop a *Hospital Management System* that simplifies the management of patient data and hospital operations through a user-friendly web application. The system is designed to support three types of users: Admin, Doctor, and Receptionist. Each user is provided with specific functionalities such as managing users, adding patients, viewing patient details, and updating diagnosis and treatment. The project utilizes modern technologies including React for the frontend, Node.js and Express for backend services, and MySQL for structured data storage.

The need for such a system is driven by the increasing demand for efficient healthcare services and accurate data management. In many small and medium healthcare institutions, manual record keeping is still prevalent, which can result in misplaced records and slower service delivery. Studies and reports on healthcare digitization indicate that implementing computerized systems can significantly reduce administrative workload, improve patient data accessibility, and enhance decision-making processes. Therefore, adopting a digital hospital management system becomes essential to meet modern healthcare requirements.

Earlier systems for hospital management were primarily desktop-based or relied on basic database applications with limited functionality. These systems often lacked scalability, user-friendly interfaces, and proper security mechanisms. With advancements in web technologies, modern systems now use client-server architecture, RESTful APIs, and role-based access control to provide better performance and security. The proposed system improves upon existing solutions by integrating a responsive user interface with a scalable backend and secure

database management, thereby offering an efficient and reliable platform for hospital operations.

1.1 Background

The healthcare sector has undergone significant transformation in recent years due to the adoption of digital technologies aimed at improving patient care and operational efficiency. Traditionally, hospitals and clinics depended heavily on manual record-keeping systems, where patient information such as personal details, medical history, prescriptions, and treatment records were maintained in paper files. Although this method was widely used, it presented several challenges including data duplication, difficulty in maintaining records, risk of physical damage or loss, and delays in retrieving critical patient information when needed.

As the number of patients visiting healthcare institutions continues to increase, managing large volumes of data manually becomes inefficient and time-consuming. Healthcare professionals often face difficulties in accessing accurate and up-to-date information, which can impact the quality of diagnosis and treatment. Additionally, coordination between different departments such as reception, doctors, and administration becomes complex without a centralized system. These limitations highlight the need for a more organized and automated approach to hospital management.

With advancements in information technology, digital systems have been introduced to replace traditional methods. A Hospital Management System (HMS) is one such solution that integrates various hospital operations into a single platform. It enables efficient storage, retrieval, and management of patient data while reducing manual workload. By providing real-time access to information, the system improves communication between healthcare staff and ensures better decision-making.

The proposed Hospital Management System in this project is developed using modern web technologies such as React for the user interface, Node.js and Express for backend processing, and MySQL for database management. The system incorporates role-based access, allowing Admin, Doctor, and Receptionist to perform specific tasks according to their responsibilities. This structured approach not only enhances data security but also improves workflow efficiency.

Overall, the transition from manual systems to a digital hospital management solution plays a crucial role in improving the quality of healthcare services. It minimizes errors, saves time,

and ensures that patient information is handled in a secure and organized manner, making it an essential tool for modern healthcare institutions.

1.2 Statistics of the internship

The need for a Hospital Management System is strongly supported by the growing demand for efficient and reliable healthcare services across the world. A large number of small and medium-sized hospitals still rely on manual or partially digital systems for managing patient records and administrative tasks. These traditional methods often lead to inefficiencies such as increased paperwork, duplication of data, and delays in accessing important medical information. Studies in healthcare digitization indicate that the use of computerized systems can significantly reduce administrative workload by approximately 30–40%, allowing hospital staff to focus more on patient care rather than routine documentation.

In many healthcare institutions, the time taken to retrieve patient records manually can range from several minutes to even hours, especially when dealing with large volumes of data. This delay can directly affect the quality of treatment and decision-making. Digital systems, on the other hand, enable instant access to patient information, improving response time and supporting timely medical interventions. Furthermore, research highlights that errors in manual data entry and record maintenance are a major concern in healthcare environments, which can be minimized through automated data handling and validation mechanisms.

1.3 Prior Existing Systems

In the early stages of healthcare digitization, hospital management was primarily supported by standalone desktop applications and basic database systems. These systems were designed to perform limited tasks such as storing patient information, generating simple reports, and maintaining basic records. In many cases, hospitals also relied on spreadsheet-based tools or partially computerized systems, where only certain operations were digitized while the rest continued manually. Although these approaches represented an improvement over completely paper-based methods, they still had several limitations that affected overall efficiency.

One of the major drawbacks of earlier systems was the lack of scalability. As the number of patients and records increased, these systems struggled to handle large volumes of data efficiently. Since most applications were installed on a single computer or within a local network, accessing information from different departments or locations was difficult. This

limited accessibility created delays in communication between receptionists, doctors, and administrative staff.

Another significant limitation was the absence of real-time data updates. In many traditional systems, data entered in one department was not immediately reflected across other sections, leading to inconsistencies and outdated information. This could negatively impact patient care, especially in situations where timely access to accurate medical records is critical. Additionally, the user interfaces of older systems were often complex and not user-friendly, requiring technical knowledge to operate effectively.

1.4 Proposed Approach

Aim of the Project

The proposed *Hospital Management System* is a web-based application designed to simplify and automate hospital operations using modern technologies such as React, Node.js, Express, and MySQL. The system follows a client-server architecture, where the frontend (React) provides an interactive user interface and communicates with the backend (Node.js and Express) through RESTful APIs. The backend processes requests, performs business logic, and interacts with the MySQL database to store and retrieve patient and user data efficiently.

A key feature of the proposed system is role-based authentication, which ensures that different users can access only the functionalities relevant to their responsibilities. The system supports three main roles:

- Admin: Has complete control over the system, including managing users (adding doctors and receptionists), viewing all patient records, and deleting records if required.
- Doctor: Can view patient details and update medical information such as diagnosis and treatment.
- Receptionist: Responsible for registering new patients and viewing existing patient records.

This role-based structure improves both security and workflow efficiency, as it prevents unauthorized access while allowing each user to perform their tasks effectively.

Motivation

The main motivation behind this project is to replace traditional manual processes with an automated system that improves accuracy, reduces workload, and enhances the overall efficiency of hospital operations. By digitizing patient records and workflows, the system aims to support faster and more reliable healthcare services.

Applications

- Patient Record Management: Efficient storage and retrieval of patient details, diagnosis, and treatment information.
- Role-Based Hospital Operations: Controlled access for Admin, Doctor, and Receptionist to perform specific tasks.
- Data Storage and Retrieval: Organized database system for handling large volumes of hospital data.
- Healthcare Workflow Optimization: Improved coordination between different departments.

Limitations

- The system currently supports only basic functionalities and does not include advanced features such as appointment scheduling, billing, or report generation.
- It requires a stable internet connection or local server setup to function properly.
- Security features are limited to basic authentication and can be enhanced with advanced encryption and authorization techniques.
- The system is designed for small to medium-scale hospitals and may require further optimization for large-scale deployment.

1.5 Objectives of the Project

The primary objective of the *Hospital Management System* is to design and develop an efficient, reliable, and user-friendly platform that simplifies hospital operations and improves patient data management. The system focuses on automation, accuracy, and secure handling of information while supporting different user roles such as Admin, Doctor, and Receptionist. The detailed objectives are as follows:

1. Behaviour

To develop an intuitive and user-friendly interface that enables users to interact with the

system easily without requiring advanced technical knowledge. The system should provide clear navigation, simple input forms, and responsive design so that Admin, Doctor, and Receptionist can perform their respective tasks efficiently. It also aims to reduce user errors by providing proper validation messages and guided workflows.

2. Analysis

To accurately collect, store, and analyze patient-related data such as personal details, diagnosis, and treatment history. The system should ensure that data is organized in a structured manner, allowing healthcare professionals to retrieve information quickly and make informed decisions. Proper data handling also supports tracking patient history over time, which is essential for effective medical care.

3. System Management

To streamline and manage hospital operations by integrating multiple functionalities into a single platform. This includes patient registration, viewing patient records, updating medical information, and managing users. The objective is to reduce manual work, eliminate redundancy, and ensure smooth coordination between different departments within the hospital.

4. Security

To implement strong security measures that protect sensitive patient information and prevent unauthorized access. The system uses role-based access control, ensuring that each user can only access features relevant to their role. For example, only doctors can update diagnosis and treatment details, while admins manage users. This objective also includes maintaining data integrity and confidentiality.

5. Deployment

To design and deploy a scalable and flexible web-based application that can be easily implemented in small to medium-sized healthcare institutions. The system should support smooth integration between frontend and backend components and allow future enhancements such as adding new features or scaling for larger environments. It should also ensure reliable performance when handling multiple users and data operations.

1.6 Sustainable Development Goals (SDGs)

The United Nations Sustainable Development Goals (SDGs) are a global framework aimed at improving health, innovation, economic growth, and reducing inequalities by 2030. The proposed **Hospital Management System (HMS)** aligns with several SDGs by improving healthcare service delivery through digital transformation, automation, and efficient data management.

SDG 3: Good Health and Well-being

The system contributes to improving healthcare services by enabling efficient management of patient records, appointments, and doctor interactions. Features such as role-based login for Doctor, Admin, and Receptionist ensure smooth coordination within the hospital. This leads to better patient care, reduced waiting time, and improved overall healthcare quality.

Doctors can quickly access patient history and treatment details, receptionists can handle patient registration and appointment booking, and administrators can manage overall hospital operations. This reduces waiting time, minimizes manual errors, and improves coordination between departments. As a result, patients receive faster and better healthcare services, contributing to improved health outcomes and overall well-being.

SDG 9: Industry, Innovation, and Infrastructure

The project promotes the use of modern technologies such as React, Node.js, Express, and MySQL to build a scalable and efficient digital system. By replacing manual processes with an automated solution, the system supports innovation in healthcare infrastructure and encourages the adoption of digital tools in hospitals.

By replacing traditional paper-based systems with an automated web-based solution, the project improves hospital infrastructure and supports technological innovation. It enables faster data processing, secure information management, and better communication between hospital departments. This digital transformation strengthens healthcare operations and encourages hospitals to adopt smart and sustainable technological solutions for future growth.

Fig 1.1 Sustainable development goals

1.7 Overview of the internship report

This project report presents the design and development of a Hospital Management System

aimed at improving the efficiency of hospital operations through digital solutions. Chapter 1 introduces the project by explaining its objectives, scope, and relevance, along with its alignment with Sustainable Development Goals. Chapter 2 reviews existing systems and related work, highlighting the limitations of traditional hospital management methods and the need for an automated system. Chapter 3 describes the methodology adopted for the project, including requirement specification, system planning, and the use of technologies such as React, Node.js, Express, and MySQL. Chapter 4 focuses on project management aspects, including the project timeline, Gantt chart, risk analysis using the PESTLE framework, and budget estimation. Chapter 5 discusses the analysis and design of the system, covering functional and non-functional requirements, system architecture, and module design such as login functionality for Doctor, Admin, and Receptionist. Chapter 6 presents the implementation details and results, demonstrating how the system works in real-time scenarios. Finally, Chapter 7 concludes the report by summarizing the outcomes of the project and suggesting possible future enhancements to further improve the system.

Chapter 2

LITERATURE SURVEY

In recent years, the healthcare industry has increasingly adopted digital systems to improve efficiency, accuracy, and accessibility of patient services. Hospital Management Systems (HMS) play a crucial role in replacing traditional paper-based records with centralized web-based applications. These systems are typically built using modern full-stack technologies such as Node.js, Express, MySQL, and React, enabling fast, scalable, and interactive solutions for hospital operations.

One of the key developments in this field is the use of role-based access control in healthcare applications. Studies in modern web systems highlight that separating user roles such as doctor, admin, and receptionist improves data security and operational efficiency. Each user is given specific permissions, ensuring that sensitive medical data is accessed only by authorized personnel. However, early systems lacked proper authentication mechanisms, leading to data privacy concerns. This limitation has encouraged the adoption of secure login systems using encrypted credentials and session management.

1.Sharma et al. proposed a **Web-based Hospital Management System** designed to improve the efficiency of hospital operations by replacing traditional manual record-keeping with a computerized platform. The main objective of their system was to create a centralized environment where patient information, doctor details, appointment records, and hospital administrative data could be stored and accessed digitally.

2.Kumar and Singh proposed a **Hospital Management System based on Client-Server Architecture** to improve communication and data sharing between different departments in a hospital. The main objective of their system was to create a structured environment where information could be transferred quickly and efficiently among departments such as reception, laboratory, pharmacy, doctor consultation, and administration. In their approach, the **server** acts as the central system that stores and manages all hospital data, while the **clients** are the individual computers or terminals used by different departments to access and update

information.

3. et al. proposed a **Cloud-based Hospital Management System** aimed at improving hospital operations through remote data storage and multi-location access. The main objective of their system was to move hospital data from traditional local storage systems to cloud platforms, allowing authorized users to access information anytime and from any location. This approach supports better management of hospital records and improves service efficiency.

4.Reddy and Kumar proposed an **Electronic Health Record (EHR) System** to improve hospital operations by maintaining patient medical history and treatment details in a digital format. The main objective of their system was to replace traditional paper-based records with an organized electronic system that allows hospitals to store, update, and retrieve patient information efficiently. This helps in improving healthcare services and reducing manual work.

5.Gupta et al. proposed a **Role-Based Access System** for hospital management to improve security and workflow efficiency by providing different access permissions to various users such as Admin, Doctor, and Receptionist. The main objective of their system was to ensure that each user could only access the information and functions related to their responsibilities, thereby protecting sensitive hospital data and improving operational control.

6.Mehta and Joshi proposed an **API-based Web Application** for hospital management to improve communication and integration between different modules of the system. The main objective of their approach was to use **REST APIs** to connect frontend interfaces, backend services, and database operations in a smooth and organized manner. This helped different hospital modules such as patient registration, doctor management, appointment scheduling, billing, and administration work together efficiently.

7.Khan et al. proposed a **User-Friendly Hospital Management System** that focused mainly on creating a simple and easy-to-use interface for hospital staff. The main objective of their system was to reduce the complexity of hospital operations by providing a clean user interface with structured forms and simple navigation. This approach helped users such as receptionists, doctors, and administrators perform daily tasks without requiring advanced technical knowledge.

8.Verma and Sharma proposed a **Distributed Hospital Management System** to improve hospital operations by enabling **multi-location data access** across different departments and branches. The main objective of their system was to allow hospitals with multiple units or departments to share information efficiently without depending on a single local system. This approach improved accessibility and coordination among staff working in different locations.

9.Singh et al. proposed a **Secure Hospital Management System with Authentication** to protect sensitive hospital data through encrypted communication and proper user authorization. The main objective of their system was to ensure that confidential information such as patient records, doctor reports, billing details, and administrative data remained safe from unauthorized access and cyber threats. Since hospital systems handle highly private information, security was considered one of the most important requirements.

10.Rao and Iyer proposed an **Integrated Hospital System** designed to manage multiple hospital operations within a single platform. The main objective of their system was to combine important hospital modules such as patient management, staff management, appointment scheduling, billing, reporting, and administrative control into one connected system. This approach helped improve hospital efficiency by reducing the need for separate software applications for different tasks.

11. Lee et al. (2017) focused on designing a **Hospital Database Management System** for storing and retrieving patient records, doctor details, staff information, and administrative data efficiently. The main objective of their system was to create a structured database that could support smooth hospital operations and reduce manual record handling. By using proper database design techniques such as normalization, the system reduced data redundancy and improved consistency across different modules.

12. Ahmed and Rahman (2019) proposed an **Automated Appointment Scheduling System** for hospitals to improve doctor scheduling and reduce patient waiting time. The main purpose of their system was to simplify the appointment booking process by allowing patients to schedule visits in an organized manner while helping hospitals manage doctor availability more effectively. The system allowed receptionists or patients to book appointments based on available time slots, reducing manual scheduling conflicts and improving hospital workflow.

Ref No	Author	Method	Key Features	Advantages	Limitation
1.	Sharma et al.	Web-based Hospital System	Centralized database for patient records	Reduced paperwork, improved accuracy	No role-based access, limited scalability
2.	Kumar & Singh	Client-Server Architecture	Data sharing between departments	Faster data processing, improved coordination	Complex interface, not user-friendly
3.	Patel et al.	Cloud-based System	Remote data storage and multi-location access	High scalability, cost-effective	Data security and privacy concerns
4.	Reddy & Kumar	Electronic Health Record (EHR)	Digital patient history and treatment tracking	Better data organization, reduced manual errors	Integration issues with different systems
5.	Gupta et al.	Role-Based Access System	Different access levels for Admin, Doctor, Receptionist	Improved security and workflow management	Complex role configuration
6.	Mehta & Joshi	API-based Web Application	Integration using REST APIs	Smooth communication between modules	Performance issues with large data
7.	Khan et al.	User-Friendly System Design	Simple UI with structured forms	Easy to use, minimal training required	Lack of advanced features and scalability

8.	Verma & Sharma	Distributed System	Multi-location data access	Better accessibility and coordination	Network dependency and latency issues
9.	Singh et al.	Secure System with Authentication	Encrypted communication and authorization	Strong data security	High computational cost
10.	Rao & Iyer	Integrated Hospital System	Combines patient, staff, and reporting modules	Strong data security	Increased system complexity

Table 2.1 Summary of Literature reviews

Chapter 3

METHDOLOGY

This chapter presents the methodology adopted for the development of the *Hospital Management System*, which is designed to manage patient records, support role-based login for Admin, Doctor, and Receptionist, and ensure efficient hospital operations. The system integrates modern web technologies such as React for the frontend, Node.js and Express for backend services, and MySQL for database management to build a reliable, scalable, and user-friendly application suitable for real-world healthcare environments.

To achieve efficient development, proper testing, and smooth deployment, the **DevOps methodology** is adopted in this project. DevOps is chosen because it supports continuous integration, continuous testing, and continuous deployment, which are essential for maintaining a dynamic web-based system. It enables better coordination between development and deployment processes, resulting in faster updates, improved performance, and higher system reliability.

As illustrated in the DevOps cycle, the methodology follows a continuous process involving planning, development, integration, testing, deployment, and monitoring. During the **planning phase**, system requirements such as patient management, user roles, and security features are defined. In the **development phase**, the application is built using React for the user interface and Node.js with Express for backend APIs. The **integration phase** ensures proper communication between frontend and backend modules through API connections.

The **testing phase** includes verifying individual components such as login functionality, patient addition, and data updates, as well as testing the complete system to ensure smooth interaction between modules. In the **deployment phase**, the application is made available for users, ensuring that all features function correctly in a real-time environment. Finally, the **monitoring phase** focuses on identifying issues, improving performance, and updating the system based on user feedback.

This continuous cycle ensures that the Hospital Management System remains efficient, secure, and adaptable to future enhancements. By adopting the DevOps methodology, the project achieves better maintainability, reduced development time, and improved overall system quality.

3.1 V-Model Methodology

The Hospital Management System follows the **V-Model Software Development Life Cycle (SDLC)**, where each development phase is directly connected to a corresponding testing phase. This model ensures better quality, systematic development, and proper validation of the system. The project includes login access for **Admin, Doctor, and Receptionist** using **React.js, Node.js, Express.js, and MySQL**.

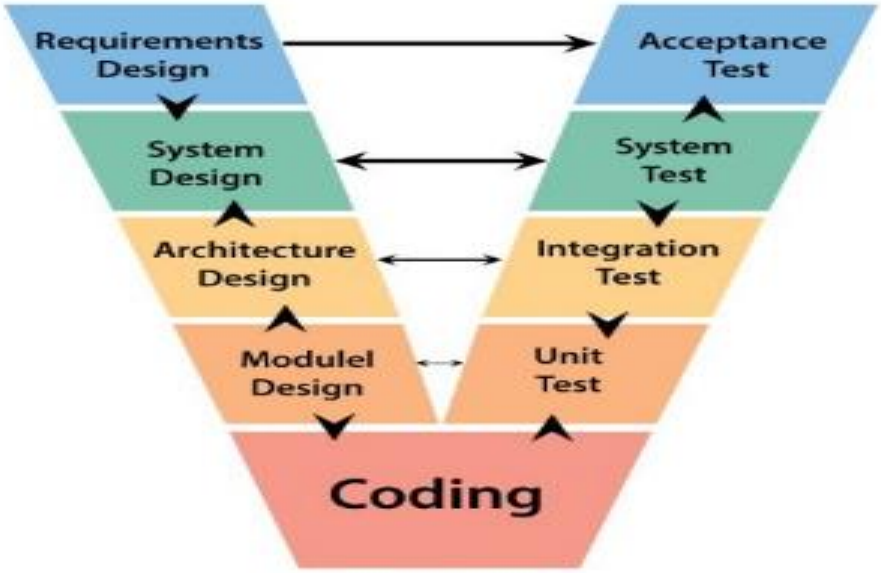


Fig 3.1 The V model methodology

1. Requirements Design (Acceptance Testing)

- In this phase, all project requirements are collected and analyzed.
- The main features include secure login for Admin, Doctor, and Receptionist, patient registration, appointment scheduling, doctor management, and report generation.
- The acceptance test verifies whether the final system satisfies all user requirements and functions correctly according to hospital needs.

2. System Design (System Testing)

- The complete system structure is designed in this stage.
- It includes frontend pages, backend architecture, database structure, user roles, and

workflow planning.

- System testing ensures that the entire Hospital Management System works properly as a complete application without errors.

3. Architecture Design (Integration Testing)

- This phase defines how different technologies interact with each other such as React frontend, Node.js backend, Express API routing, and MySQL database connectivity.
- Integration testing checks whether all modules communicate correctly and data flows smoothly between frontend and backend.

4. Module Design (Unit Testing)

- Individual modules are designed separately such as Login Module, Doctor Module, Admin Module, Reception Module, Patient Registration Module, and Appointment Module.
- Unit testing verifies each module independently to ensure every function works correctly before combining them.

5. Coding

This is the implementation phase where actual programming is done using:

- **Frontend:** React.js
- **Backend:** Node.js + Express.js
- **Database:** MySQL
- **Authentication:** Login validation for Admin, Doctor, Receptionist

6. Unit Testing

- Each module is tested separately, such as checking login validation, patient registration form, and appointment booking functionality.

7. Integration Testing

- After unit testing, modules are combined and tested together to verify proper communication between frontend, backend, and database.

8. System Testing

- The complete hospital management application is tested as a whole to ensure

performance, reliability, and security.

9. Acceptance Testing

- Final testing is performed by users or project evaluators to confirm that the system meets all project objectives and hospital requirements.

Fig3.2 The Devops methodology

3.1.1 Phases of Development and Testing (DevOps Mapping)

Phases	Project Stage	Activities / Deliverables	Tools Used
Planning (Requirements)	Problem Identification & Requirement Gathering	Understanding the need for a centralized hospital system. Identifying user roles such as Doctor, Admin, and Receptionist.	Requirement documents, online research
Analysis	Feasibility Study & Requirement Analysis	Analyzing current manual and digital hospital systems. Evaluating system feasibility in terms of cost, performance, and scalability.	Documentation tools, system analysis techniques
System Design	Architecture Design	Designing the overall architecture of the application	Draw.io, UML diagrams

		including frontend, backend, and database layers.	
Functional Design	Module-Level Design	Designing specific functionalities like authentication system (Doctor/Admin/Reception login), patient data handling, appointment scheduling, and role-based access control.	React (UI design), Node.js planning
Development (Implementation)	Application Development	Building the frontend using React for dynamic UI. Developing backend APIs using Node.js and Express. Creating and managing database schema using MySQL.	React (UI design), Node.js planning
Continuous Testing	Testing Modules	Performing unit testing for individual components and integration testing for combined modules.	Manual Testing
Integration	System Integration	Connecting frontend with backend APIs and database. Ensuring smooth communication between modules such as login, patient records, and appointment scheduling.	REST APIs, Express
Deployment	System deployment	Deploying the application on a local server or cloud platform. Making the system accessible through a web browser with proper configuration.	Node server, cloud platforms (optional)

Monitoring & Feedback	Performance Evaluation	Monitoring system usage, identifying bugs, and improving system performance.	User feedback, logging tools

Table 3.1 Phases of DevOps Model

3.3 Project Management Methodology

The development of the Hospital Management System (HMS) follows a suitable project management methodology to ensure proper planning, execution, testing, and successful deployment. Since the project includes multiple modules such as Admin Login, Doctor Login, Reception Login, Patient Registration, Appointment Scheduling, and Database Management, selecting the right methodology is important for smooth development.

Agile Methodology

The most suitable approach because it supports flexibility, continuous improvement, and faster development cycles. Since hospital requirements may change during development, Agile allows regular updates and improvements without affecting the complete system.
The project is divided into smaller modules such as login management, doctor management, patient records, and appointment scheduling. Each module is developed, tested, and improved step by step.

Advantages of Agile Methodology

1. Flexibility and Adaptability
- Changes in hospital requirements can be easily handled during development.
- New features such as additional user roles or report generation can be added without major difficulty.

2. Better Team Collaboration
- Frontend, backend, and database development can be managed together with continuous communication between development stages.

3. Continuous Testing and Feedback
- Each module is tested after development, reducing the chances of major system failure at the final stage.

4. Faster Delivery
- Important modules like login authentication and patient registration can be

completed early and improved later.

5. Improved Software Quality

- Frequent testing and updates improve reliability, security, and overall system performance.

Disadvantages of Agile Methodology

1. Less Predictability

- Since requirements may change frequently, project timelines may sometimes become difficult to estimate.

2. Requires Skilled Team Members

- Developers must have proper knowledge of React, Node.js, Express, and MySQL for successful implementation.

3. Difficult for Large-Scale Projects

- If the hospital system becomes very large with many departments, Agile may require stronger project control.

3.4 Project Breakdown to Task

The overall project breakdown of the Hospital Management System (HMS) into four major phases: Concept, Development, Execution, and Transfer. This structured division helps in managing the project efficiently by organizing tasks according to their importance and development sequence. The system is developed using React.js for frontend, Node.js and Express.js for backend, and MySQL for database management, with role-based login access for Admin, Doctor, and Receptionist.

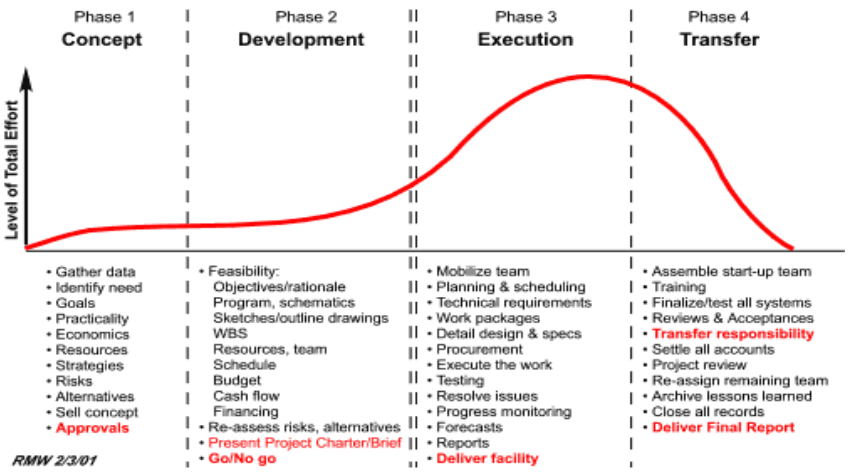


Fig 3.4 Summary of project breakdown to task

Phase 1: Concept

- The concept phase is the initial stage of the project where the problem is identified and the project objectives are clearly defined.
- In this phase, the need for a digital hospital management system is analyzed to replace manual record handling and improve hospital operations.
- Requirements such as secure login, patient registration, appointment scheduling, doctor management, and receptionist access are collected.

Phase 2: Development

- The development phase focuses on system design and module creation. The database structure is designed using MySQL, and frontend interfaces are planned using React.js. Backend APIs are developed using Node.js and Express.js to manage communication between the user interface and the database.

Phase 3: Execution

- In the execution phase, all developed modules are integrated to form the complete Hospital Management System.
- The frontend, backend, and database are connected to ensure smooth system functionality. Testing activities such as unit testing, integration testing, and system testing are performed to identify and correct errors.

Phase 4: Transfer

- The transfer phase is the final stage where the completed system is prepared for deployment and user acceptance.
- Documentation, final testing, and project report preparation are completed in this stage. The system is handed over for demonstration and evaluation.

Chapter 4

PROJECT MANGEMENT

4.1 Internship timeline

This chapter outlines the planned timeline followed during the development of the Hospital Management System, which was carried out as part of the internship. The timeline reflects a systematic approach used to design, develop, and implement the system, ensuring that all phases were completed efficiently within the allocated duration.

The project was organized into multiple stages, beginning with requirement gathering and progressing through system design, development, testing, and final deployment. Each stage included clearly defined objectives, tasks, and expected outcomes. Special focus was given to implementing core features such as secure login functionality for Doctor, Admin, and Receptionist, patient record management, and appointment handling using modern web technologies like React, Node.js, Express, and MySQL.

Initially, time was allocated to understanding the problem domain and analyzing existing hospital systems. This was followed by designing the system architecture, including frontend and backend interaction as well as database structure. During the development phase, the user interface was built using React, while backend services and APIs were developed using Node.js and Express. The database was structured and managed using MySQL to handle user data and hospital records efficiently.

The project execution was continuously monitored through regular progress checks and milestone evaluations. Each phase was carefully scheduled to ensure smooth transition between tasks and to avoid delays. Testing was performed alongside development to verify the functionality of modules such as login authentication, data management, and system integration.

A Gantt chart (Figure 4.1) was used to visually represent the timeline of the project. It illustrates the sequence of activities, their duration, start and end periods, and completion status. This chart helped in tracking progress, managing time effectively, and ensuring that all deliverables were achieved as planned.

The major phases highlighted in the timeline include requirement analysis, system design, frontend and backend development, database implementation, testing, and deployment. Each activity is represented over a specific duration, providing a clear overview of the workflow followed throughout the internship.

Table 4.1 Project planning timeline

Major Task	W1	W2	W3	W4	W5	W6	W7	W8	W9	W10	W11	W12	W13	W14	W15
Project initiation (i)															
Selection of topic															
Background (ii)															
Objectives (iii)															
Methodology (iv)															
Literature review (v)															
Design and Analysis															
System Requirement Phase (vi)															
System design phase (vii)															
Functional unit design phase (viii)															
Report															
Final report															

1. Project Initiation (i)
   - The project begins with the initiation phase.
   - Topic selection is carried out during Week 1 to Week 2.
   - This phase helps in identifying the project scope and defining the main problem to be solved.
2. Background (ii)
   - The background study is completed during Week 2 to Week 3.
   - It includes understanding the existing system, identifying problems, and studying the need for the proposed system.
3. Objectives (iii)
   - Project objectives are defined during Week 3 to Week 4.
   - This phase focuses on setting the goals, expected outcomes, and purpose of the project.
4. Methodology (iv)
   - The methodology is planned during Week 4 to Week 5.
   - It explains the methods, tools, technologies, and development approach used for the project.
5. Literature Review (v)
   - Literature review is conducted during Week 5 to Week 8.
   - This includes studying previous research papers, existing systems, and related technologies.
6. System Requirement Phase (vi)
   - Requirement analysis is carried out during Week 6 to Week 9.
   - Functional and non-functional requirements of the system are identified and documented.
7. System Design Phase (vii)
   - System design is performed during Week 7 to Week 10.
   - This phase includes designing architecture, database structure, UML diagrams, and

interface planning.

8. Functional Unit Design Phase (viii)
   - Functional unit design is completed during Week 8 to Week 12.
   - Individual modules such as admin, doctor, patient, appointment, billing, and report generation are designed.
9. Final Report
   - The final report preparation takes place during Week 10 to Week 14.
   - It includes documentation of all chapters, testing results, implementation details, and project conclusions.

Major Task	W1	W2	W3	W4	W5	W6	W7	W8	W9	W10	W11	W12	W13	W14	W15
Simulation															
Unit															
Integrated															
Hardware implementation															
Software															
Testing *															
Critical Evaluation **															
Social, Ethical, Legal, and Sustainability															
Report															
Final report															

Table 4.2 Project implementation timeline

The project implementation timeline provides a well-organized view of how the **Hospital Management System** was developed during the internship period. It highlights the step-by-step execution of the project, starting from requirement analysis to final deployment. The system mainly focuses on functionalities such as secure login access for **Doctor, Admin, and Receptionist**, along with efficient management of patient records and appointments using **React, Node.js, Express, and MySQL**.

The development process was divided into multiple phases, including requirement gathering, system design, application development, integration, testing, and deployment. Each phase was carefully scheduled and aligned with internal review milestones to monitor progress and ensure timely completion of tasks.

4.2 Risk Analysis

Risk analysis is an important aspect of project management that helps in identifying potential challenges that may affect the successful development and deployment of the **Hospital Management System**. It involves examining both internal and external factors that could impact system performance, security, and usability.
To systematically evaluate risks, the **PESTLE framework** is used. This framework considers six major factors—Political, Economic, Social, Technological, Legal, and Environmental—that may influence the project.

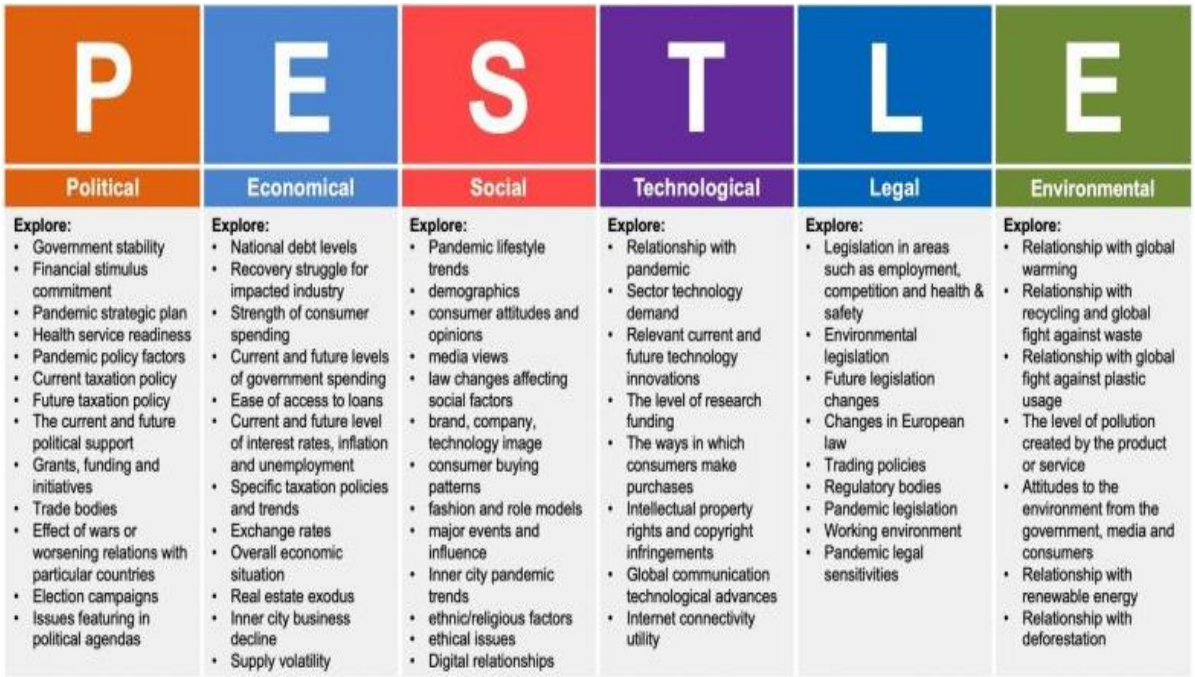


Fig 4.1: PESTLE-Based Risk Analysis

The PESTLE analysis for the Hospital Management System, highlighting various external factors that could influence the project. Each factor is associated with its potential impact and corresponding mitigation strategy to reduce risks.
By considering these factors during development, the system becomes more reliable, secure, and adaptable to real-world conditions. This proactive approach helps in minimizing uncertainties and ensures the successful implementation of the Hospital Management System.

Factor	Description
Political	Government healthcare policies, hospital regulations, digital health initiatives, funding support, and public

	healthcare programs influence the development of the Hospital Management System. Compliance with government hospital standards is necessary for smooth implementation.
Economical	Budget availability for software development, cost of system maintenance, hardware expenses, staff training costs, and long-term financial benefits through automation affect the project. The system helps reduce paperwork and operational costs.
Social	Patient expectations for faster service, staff willingness to adopt digital systems, user-friendly interfaces, healthcare awareness, and improved patient satisfaction play an important role. Easy access for Admin, Doctor, and Receptionist improves hospital workflow.
Technological	Use of modern technologies such as React.js, Node.js, Express.js, and MySQL supports efficient system development. Internet availability, system security, database performance, API integration, and scalability are major technological factors.
Legal	Patient data privacy laws, healthcare regulations, login security requirements, data protection policies, and hospital compliance standards must be followed. Secure authentication for Admin, Doctor, and Receptionist is essential.
Environmental	Digital record management reduces paper usage and supports eco-friendly hospital operations. Online appointments and digital reports help minimize physical documentation and contribute to sustainable healthcare management.

Table 4.2 PESTLE-Based Risk Analysis

1.Political Factors : Political factors have a strong influence on the development and implementation of a Hospital Management System because healthcare services are closely connected with government policies and regulations. Government healthcare policies determine how hospitals manage patient records, appointments, billing, and treatment processes. Public health initiatives such as vaccination programs, disease monitoring, and health awareness campaigns may also require integration with the hospital system for proper tracking and reporting.

2. Economical Factors: Economic factors play an important role in the successful implementation of the Hospital Management System. The cost of software development, hardware installation, database management, system maintenance, and staff training directly

affects project planning. Hospitals, especially government and small private hospitals, often work within limited budgets, so cost-effective solutions are necessary. The use of technologies such as React.js, Node.js, Express.js, and MySQL helps reduce development costs while maintaining system efficiency.

3. Social Factors: Growing demand for digital healthcare services (online appointments, records) . Patient expectations for quick and efficient service . Awareness of health and hygiene after pandemics. Demographic factors (aging population increases system usage). Trust in digital systems and data privacy concerns.

4. Technological Factors: Use of AI for diagnosis, scheduling, and predictions Integration with electronic health records (EHR) systems Cloud computing for data storage and accessibility. Cybersecurity measures to protect sensitive patient data Adoption of telemedicine and mobile health applications

5. Legal Factors: Legal factors play an important role in the development of the Hospital Management System because hospitals handle highly sensitive patient information and must follow strict healthcare regulations. The system must comply with healthcare laws and data protection policies to ensure that patient records are stored and managed securely. Patient privacy is a major concern, and access to medical records should only be allowed to authorized users such as Admin, Doctor, and Receptionist based on their roles.

6. Environmental Factors: Environmental factors influence the Hospital Management System by supporting sustainable and eco-friendly healthcare practices. One of the major benefits of the system is the reduction in paper usage through digital record management, which helps minimize physical documentation and supports environmental conservation. Patient registration, appointment scheduling, prescriptions, and reports can all be managed electronically, reducing waste and improving efficiency.

4.3 Internship budget

The Hospital Management System developed during the internship was implemented using freely available and open-source technologies, resulting in no direct financial cost. All stages of the project, including design, development, testing, and deployment, were carried out using a personal computer and standard development tools. This eliminated the need for additional infrastructure or paid software licenses.

The system was built using Node.js and Express.js for backend development, React for the frontend interface, and MySQL for database management. These technologies are open-source and widely supported, making them suitable for developing scalable and efficient web applications. The implementation of role-based login features for doctors, administrators, and receptionists was achieved without any additional cost by utilizing built-in authentication techniques and libraries.

The use of open-source frameworks provided flexibility in designing and customizing the system according to the project requirements. Since there were no licensing or subscription costs, the overall solution remains highly cost-effective and suitable for small and medium healthcare organizations.

Table 4.3 Internship Project Budget Details

Item	Resource Used	Cost (INR/USD)
Development Environment	Node.js & Express.js (Open-source backend framework)	₹0 / $0
Frontend Technology	React native	₹0 / $0
Database System	MySQL (Open-source relational database)	₹0 / $0
Authentication System	Role-based Login (Doctor, Admin, Reception)	₹0 / $0
API Development system	RESTful APIs using Express.js	₹0 / $0
Development Tools	Visual Studio Code (Free IDE)	₹0 / $0
Version Control	Git & GitHub (Free tier)	₹0 / $0
Testing Platform	Localhost / Browser Testing	₹0 / $0
Documentation	Digital documentation / minimal printing	₹0 / $0

Chapter 5

Analysis and Design

Analysis and design are two closely related phases in system development. The analysis phase focuses on understanding the problem, identifying user needs, and gathering detailed requirements. It defines what the system should do. The design phase, on the other hand, concentrates on creating a structured solution by defining system architecture, components, and workflows.

In the context of the Hospital Management System, analysis involves studying hospital operations such as patient registration, appointment scheduling, and role-based access for Doctor, Admin, and Receptionist. The design phase translates these requirements into a working system using technologies like React, Node.js, Express, and MySQL, ensuring efficiency, usability, and security.

5.1 Requirements

The design and implementation of the AI-based real-time object detection, measurement, and weight estimation system are based on a clear understanding of both functional and non-functional requirements to ensure accuracy, efficiency, and reliability. The system is developed to perform real-time analysis using image and video input while maintaining user-friendly interaction and consistent performance.

Functional Requirements

1. The system shall provide secure login functionality for different user roles, including Doctor, Admin, and Receptionist.
2. The system shall allow administrators to add, update, and manage user accounts and system data.
3. The system shall enable receptionists to register new patients and manage patient details.
4. The system shall allow doctors to view patient records and update medical information.
5. The system shall support appointment scheduling, including booking, updating, and canceling appointments.
6. The system shall store and retrieve patient and appointment data efficiently using a MySQL database.
7. The application shall provide dashboards for each user role to access relevant information quickly.
8. The system shall allow data modification operations such as create, read, update, and delete (CRUD).

Non-Functional Requirement

1. **Performance:** The system should respond quickly to user requests, ensuring smooth login, data retrieval, and updates without noticeable delay.
2. **Scalability:** The system should be capable of handling increasing numbers of users, patient records, and transactions as the hospital grows.
3. **Security:** User authentication must be secure, with proper validation of login credentials and protection of sensitive patient data through access control mechanisms.
4. **Usability:** The interface should be simple, intuitive, and easy to navigate for all users, including non-technical staff such as receptionists.
5. **Reliability:** The system should operate consistently without failures and ensure accurate storage and retrieval of data. (YOLOv8) should achieve high detection accuracy on the test dataset, with reliable bounding box predictions and confidence scores. Measurement and weight estimation should provide approximate but consistent results under controlled conditions.

Category	**Description**
Purpose	The system is developed to manage hospital operations digitally, enabling efficient handling of patient records, appointments, and role-based login access for Doctor, Admin, and Receptionist.
Behaviour	The system supports multiple user roles with different functionalities. Admin manages overall system data, Receptionist handles patient registration and appointments, and Doctor views and updates patient records.
System Management	The system provides centralized control for managing users, patient data, and appointments. Admin users can monitor system activities and maintain data integrity.
Data Analysis	The system organizes and processes patient and appointment data to provide meaningful information such as patient history and appointment status.
Application Deployment	The application is deployed on a local server or cloud environment and can be accessed through a web browser by authorized users.

Table 5.1: Summarizing Requirements of Hospital Management System

5.2 Block Diagram

The **Hospital Management System** is designed as a web-based application that integrates frontend, backend, and database components to efficiently manage hospital operations. The system follows a structured flow consisting of input, processing, and output stages, ensuring smooth interaction between users and the system. Key functionalities include secure login for **Doctor, Admin, and Receptionist**, patient management, and appointment scheduling.

Data Flow Description

1.User Input: The system begins with user interaction through the web interface developed using React. Users such as Doctor, Admin, and Receptionist enter their login credentials and provide necessary data such as patient details and appointment information.

2.Authentication Process: The entered login credentials are sent to the backend server built with Node.js and Express. The system verifies the user details by checking them against the stored data in the MySQL database.

3.Access Control: Once authenticated, the system grants role-based access. Each user (Doctor, Admin, Receptionist) is directed to their respective dashboard with permitted functionalities.

4.**Data Processing:** The backend processes user requests such as adding patient records, updating details, scheduling appointments, and retrieving information. Business logic is applied to ensure data consistency and correctness.

5.**Database Interaction:** All data is stored and managed in the MySQL database. The backend communicates with the database to perform operations like insert, update, delete, and fetch records.

6**.Output Display:** The processed information is sent back to the frontend, where it is displayed in a user-friendly format. Users can view patient details, appointment schedules, and other relevant data through dashboards.

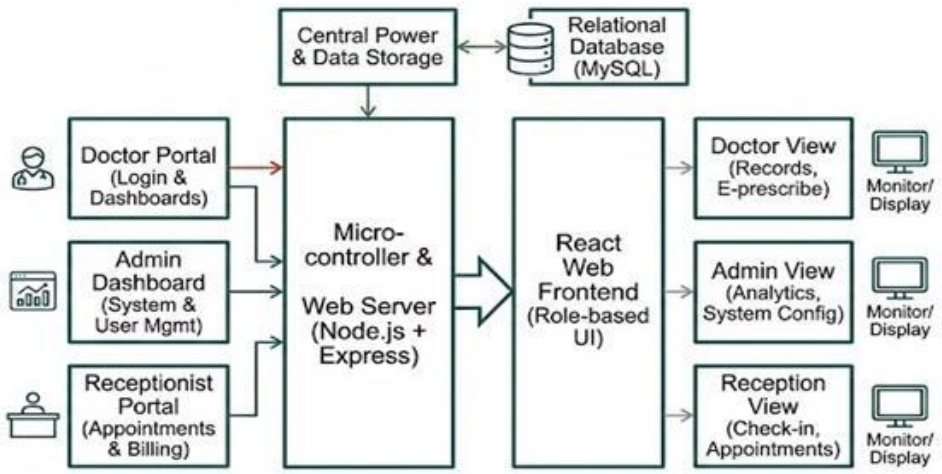


Figure 5.1: Block Diagram

5.3 System Flow

The operational architecture of the **Hospital Management System** is designed to ensure a secure, role-based transition from the initial server handshake to end-user task execution. The flow integrates a **React-based frontend** with a **Node.js/Express backend**, utilizing **MySQL** for persistent data storage.

5.3.1 Initialization and Authentication

The process initiates when the environment variables are loaded and the **Node.js** server establishes a connection pool with the **MySQL database**.

1. **Entry Point:** The user interacts with the **React** interface, landing on a unified Login Gateway.
2. **Credential Submission:** Users (Admins, Doctors, or Receptionists) submit their unique identifiers.
3. **Backend Validation:** The **Express** router intercepts the POST request, querying the database to verify the user's existence and encrypted password.
    - **Failure State:** If credentials mismatch, a **401 Unauthorized** error is returned to the React frontend, triggering a toast notification for the user to retry.
    - **Success State:** Upon validation, a **JSON Web Token (JWT)** or session is generated, containing the user's specific role.

5.3.2 Role-Based Redirection (RBAC)

Once authenticated, the system evaluates the user's role to determine the accessible logic and UI components:

- **Administrative Flow:** Grants access to the Admin Dashboard, enabling CRUD (Create, Read, Update, Delete) operations on staff profiles, department configurations, and system-wide audit logs.
- **Clinical Flow (Doctor):** Routes the user to the Doctor Portal. Here, the flow focuses on data retrieval—fetching patient histories and diagnostic records—and data entry for new prescriptions or clinical notes.
- **Operational Flow (Receptionist):** Directs the user to the Reception Desk UI, optimized for high-frequency tasks such as new patient onboarding and real-time appointment scheduling.

5.3.3 Data Interaction and Persistence

Every action performed within a dashboard follows a standard request-response cycle:

1. **Request:** User performs an action (e.g., "Add Patient").
2. **Processing:** Node.js executes business logic and validates the input data against the **Requirement Specifications**.
3. **Persistence:** The system executes an **SQL query** to update the MySQL tables.
4. **Feedback:** The system sends a success response back to the React frontend, which updates the state and refreshes the view for the user.

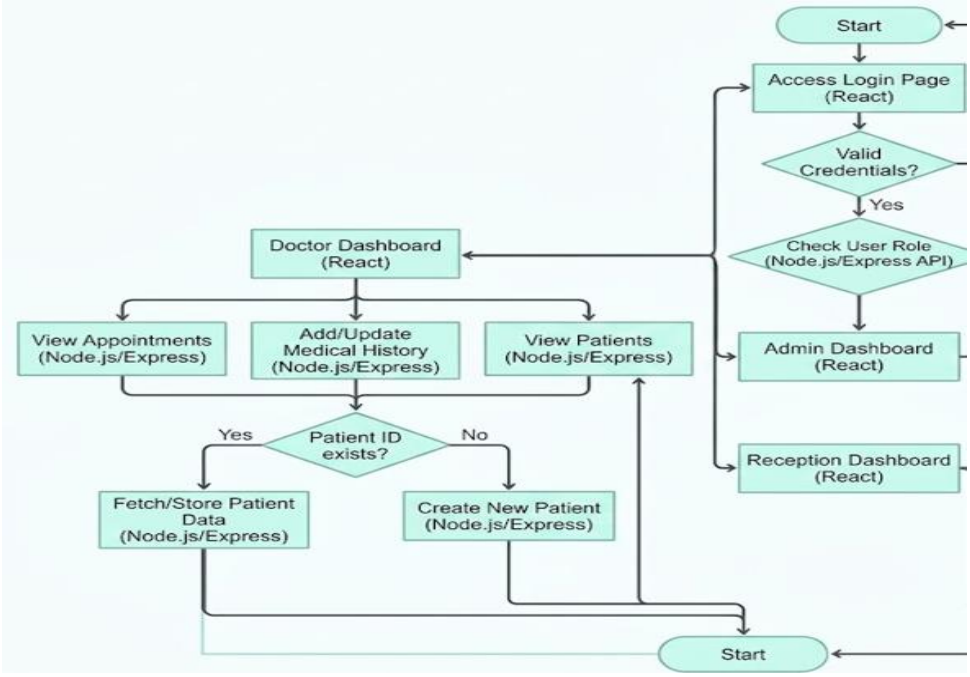


Fig 5.2 System flow chart

Functional Hardware Unit Design Phase

1. Identify hardware components (Computer systems, servers)
2. Study hardware datasheets (Processor, RAM, storage)
3. Compare components for performance and cost
4. Design system architecture
5. Develop unit testing plan for hardware reliability

Functional Software Unit Design Phase

1. Identify software components:
   - Frontend: React
   - Backend: Node.js, Express
   - Database: MySQL
2. Study software requirements and compatibility
3. Design modules (Login, Patient Management, Appointment System)
4. Analyze system performance and scalability
5. Develop unit test plans for each module

5.4 Device Selection

This project is designed as a **web-based Hospital Management System**, and it does not rely on specialized hardware components such as sensors or embedded devices. Instead, the system requires standard computing devices to support development, testing, and deployment of the application. The device selection focuses on ensuring reliable performance, smooth user interaction, and efficient handling of backend operations.

Hardware Used

- **Laptop/ Desktop System** : A computer system with at least an Intel Core i5 processor (or equivalent) and 8 GB or more RAM was used for developing and running the application. This system supports both frontend (React) and backend (Node.js, Express) processes efficiently.
- **Display Unit (Monitor)**: A standard display screen is used to interact with the system interface, including login pages and dashboards for Admin, Doctor, and Receptionist.
- **Input Devices (Keyboard and Mouse)** : These are used for entering login credentials, managing patient records, and performing administrative tasks within the system.
- **Network Connectivity** : Internet or local network connectivity is required for communication between client and server, especially when the backend and database are hosted separately.

Software Environment (Supporting Setup)

- **Frontend**: React.js for building interactive user interfaces
- **Backend**: Node.js with Express framework for handling API requests
- **Database**: MySQL for storing user credentials, patient data, and system records
- **Development Tools**: VS Code or similar IDE for coding and debugging

Selection Rational

- **Efficient Processing**: A system with moderate processing power ensures that the application runs smoothly without lag during login authentication and data operations.
- **Multi-User Support**: The selected setup supports multiple roles (Admin, Doctor, Receptionist) accessing the system simultaneously without performance degradation.
- **Ease of Development**: Standard laptops or desktops provide a flexible environment for developing full-stack applications using modern technologies.
- **Reliability**: Using widely available hardware ensures system stability and reduces dependency on specialized devices.
- **Scalability**: The system can be deployed on more powerful machines or cloud servers in the future to support larger hospital environments and increased user load.

5.5 Designing Units

The Hospital Management System is organized into multiple functional units to simplify development, testing, and maintenance. Since the project is a **software-based web application**, the design focuses on logical modules rather than physical circuits. Each unit is responsible for a specific functionality and communicates with other units through well-defined interfaces such as APIs.

1. User Authentication Unit (Login Module)

- Handles login for **Admin, Doctor, and Receptionist**
- Accepts username and password from the user
- Verifies credentials using backend services

- Generates session/token for authorized access

2. Frontend Unit (React Interface)
- Provides user interface for login and dashboards
- Displays forms, patient data, and system outputs
- Sends requests to backend using HTTP/REST APIs

3. Backend Processing Unit (Node.js + Express)
- Handles business logic and request processing
- Validates user input and manages routing
- Communicates with the database

4. Database Unit (MySQL)
- Stores user credentials, roles, patient records, and appointments
- Performs CRUD (Create, Read, Update, Delete) operations
- Ensures data consistency and security

5. Role-Based Access Unit
- Differentiates access for Admin, Doctor, and Receptionist
- Restricts unauthorized operations
- Ensures secure navigation between modules

5.6 Standards

The Hospital Management System is developed by following widely accepted software engineering, web security, and data management standards to ensure that the system is reliable, secure, and maintainable. Even though this project is implemented as part of an internship, standard practices are incorporated to achieve industry-level quality and usability. Standards play an important role in ensuring consistency, interoperability, and protection of sensitive data, especially in applications like hospital systems where user and patient information must be handled securely.

Table 5.2 Standards Adopted Across Domains

Domain	**Standard Adopted**	**Purpose**
Data Security	ISO/IEC 27001	Ensures secure storage and handling of user and hospital data

Software Development	SDLC(Software Development Life Cycle)	Provides a structured approach for design, development, and testing
Web Security	OWASP Top 10	Protects against common web vulnerabilities such as SQL injection and XSS
Data Exchange	JSON Format	Enables efficient communication between frontend and backend
Communication	HTTP/HTTPS Protocol	Facilitates secure data transfer between client and server
Data Privacy	Basic Data protection practices	Ensures safe handling of login credentials and user inputs

5.7 Domain Model Specification

The domain model represents the key concepts, entities, and their relationships within the Hospital Management System. It provides a high-level view of how different components of the system interact with each other. This model is independent of specific technologies, although it is implemented using React, Node.js, Express, and MySQL in this project.
The domain model helps in identifying the core elements such as users, patient records, and system services, along with their interactions.

Table 5.3 Description of Domain model

Domain Element	Description
Physical Entity	Represents real-world objects such as patients, doctors, and hospital staff. These are the primary entities about which the system stores and manages information.
Virtual Entity	Digital representation of physical entities. For example, patient records, doctor profiles, and login credentials stored in the database act as virtual entities corresponding to real individuals.
Device	Devices such as computers or laptops used by Admin, Doctor, and Receptionist to access the system. These devices act as an interface between users and the software system.

Resource	Software components that support system functionality. This includes the MySQL database (for storing data), backend APIs (Node.js, Express), and frontend components (React).
Service	Functional operations provided by the system such as login authentication, patient registration, record management, and appointment handling. These services interact with resources to perform tasks.

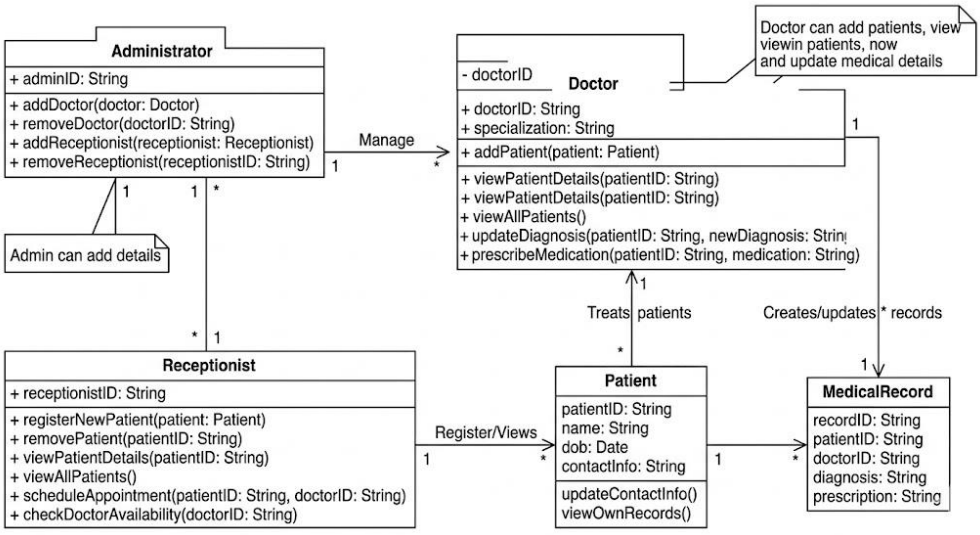


Fig.5.3 UML Class Diagram Representation

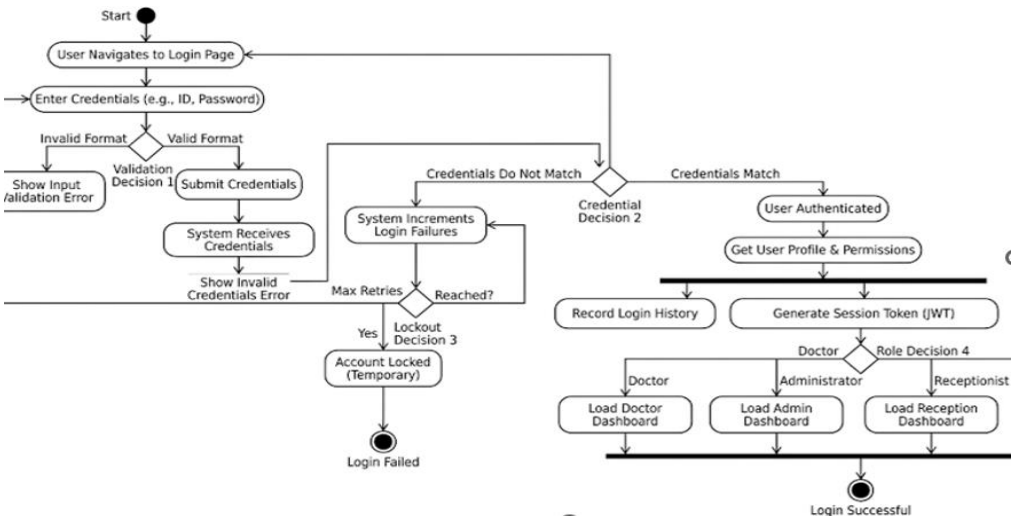


Fig.5.4 Activity Diagram Representation

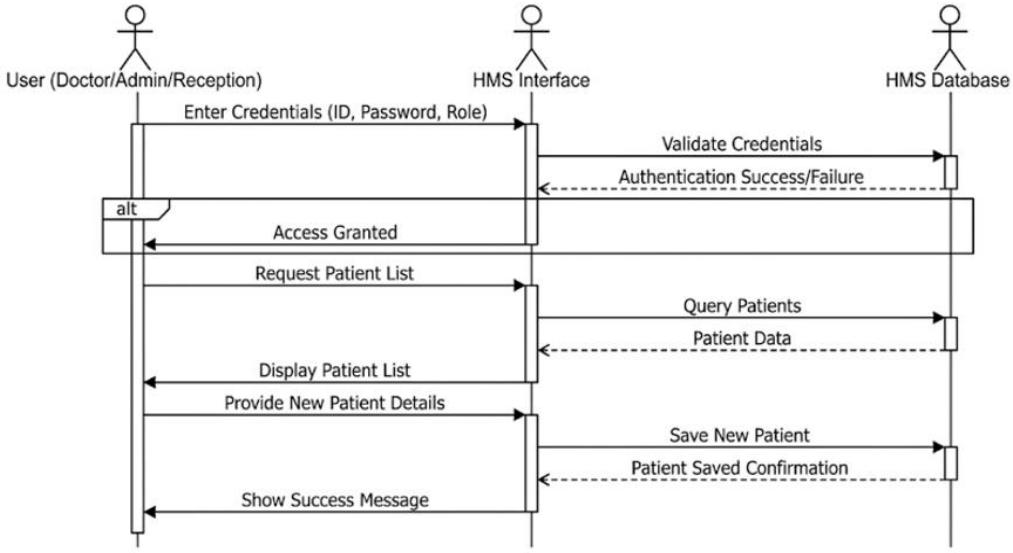


Fig.5.5 Sequence Diagram Representation

5.9 Communication Model

The Hospital Management System follows a **client–server communication architecture**, which enables efficient interaction between users and the system. This model ensures smooth data exchange, fast processing, and secure handling of information.

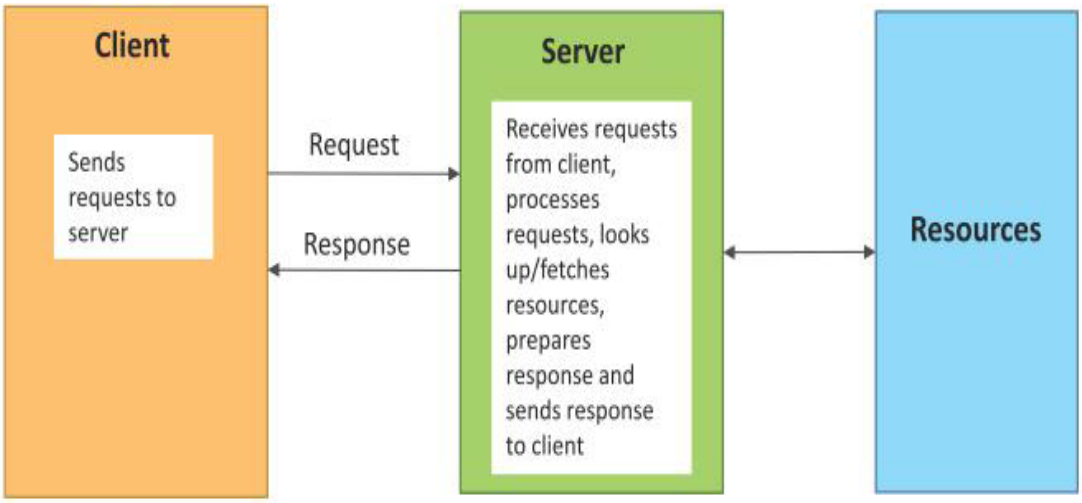


Fig 5.4 Communication model

1.Client Side (Frontend Layer)

- The client side of the system is developed using **React**, which provides a responsive and user-friendly interface.
- It acts as the interaction layer where users such as **Admin, Doctor, and Receptionist** access the system.
- The login page allows users to enter their credentials securely. After authentication, users are redirected to their respective dashboards.
- The interface supports operations such as patient registration, viewing records, and managing data.
- It captures user inputs and sends requests to the backend server.It also displays responses such as success messages, errors, and retrieved data in a structured format.

2.Server Side (Backend Layer)

- The server side is implemented using **Node.js with Express**, which handles all application logic and request processing.
- It receives requests from the React frontend.
- Validates user inputs such as login credentials. Processes business logic related to user roles (Admin, Doctor, Receptionist).
- Interacts with the **MySQL database** to store and retrieve data.

3.Communication Protocol

- Communication between the client and server is carried out using HTTP/HTTPS protocols.
- HTTP is used for general data exchange between frontend and backend.
- HTTPS ensures secure communication by encrypting sensitive data such as login credentials.

4.Data Format

- The system uses **JSON (JavaScript Object Notation)** as the standard format for communication.
- Data such as login details, patient records, and responses are exchanged in JSON format.
- JSON is lightweight, easy to read, and simple to process in both frontend and backend.
- It ensures consistency and smooth data transfer across system components.

5.10 Functional View

The functional view represents the major operations carried out by the system:

- **Input Acquisition:** The system begins by collecting input from users through the web interface. Users such as **Admin, Doctor, and Receptionist** enter data like login credentials, patient details, and appointment information. The login page captures username and password Forms are used to input patient and hospital-related data **.**The system supports structured input to ensure accuracy

- **Processing**
  Once the input is received, it is processed by the backend system developed using **Node.js and Express**. User credentials are validated for authentication Role-based access is determined (Admin, Doctor, Receptionist) . Business logic is applied to handle operations such as patient registration, updates, and record management. Requests are routed to appropriate services.

- **Data Management**
  During processing, the system interacts with the **MySQL database** to manage data. Stores user login details and roles . Maintains patient records and appointment data. Retrieves required information based on user requests. Updates existing records when changes are made Efficient database handling ensures data consistency and reliability.

- **Output Generation**
  After processing, the system generates appropriate outputs and sends them back to the user interface. Displays dashboards based on user roles. Shows patient details, appointment status, and system messages. Provides feedback such as login success or error notifications .The output is presented in a clear and organized format using the React frontend.

- **Storage & Retrieval**
  The system supports storing and retrieving information for future use.All data is

securely stored in the MySQL database. Users can access previously stored patient records and system data. Enables easy tracking, updating, and reporting of hospital activities . This functionality helps maintain a structured and accessible data system.

5.11 Operational View

The operational view explains how different components of the system interact during execution:

1. Initialization
At the beginning, all system components are prepared. The **Node.js server** is started using Express. Connection to the **MySQL database** is established. Configuration settings are loaded.

2. Execution
This stages how users interact with the system. Users (Admin, Doctor, Receptionist) access the system through the React web interface. They enter login credentials or perform actions. The frontend sends an HTTP request to the backend server.
3. Processing
After receiving the request, the backend performs several operations. Validates login credentials (authentication). Checks user role (authorization).Executes required business logic. Communicates with the **MySQL database** to fetch or update data.

4. Monitoring
The system continuously observes its performance. Tracks request-response flow. Logs errors and system activities. Monitors server performance and security. Ensures the system runs smoothly without failures.

5. Maintenance
After deployment, the system is improved over time. Updates and new features are added. Bugs and errors are fixed. Database backup and optimization are performed. Performance improvements are applied

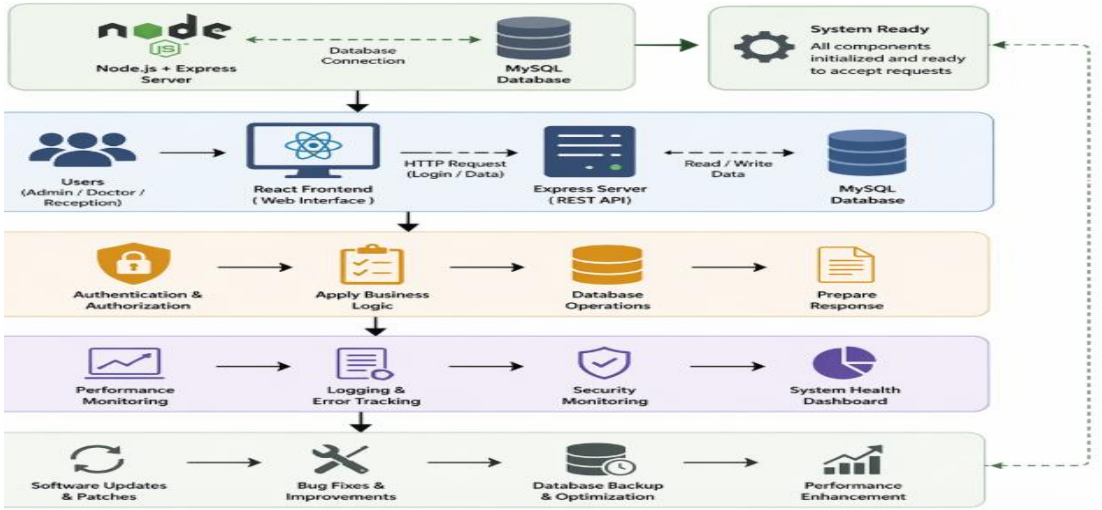


Fig 5.5 Operational view

5.12 Other Design Considerations

User Interface Design

The system provides a **clean and easy-to-use interface** developed using React. The login page is simple and allows secure access for Admin, Doctor, and Receptionist. Dashboards are designed based on user roles for better clarity and usability. Forms for patient registration and data entry are structured and easy to navigate.

System Design

The Hospital Management System follows a **modular architecture**. Different functionalities such as login, patient management, and database operations are divided into separate modules. Each module works independently but is connected through the backend. This approach improves maintainability and allows easy updates or feature additions.

API Design

The communication between frontend and backend is handled through **RESTful APIs** built using Node.js and Express. APIs handle requests such as login validation, data retrieval, and updates. Data is exchanged in **JSON format** for efficiency .Endpoints are designed to provide quick responses and smooth interaction .

Error Handling

The system includes effective error handling mechanisms to maintain stability. Input validation is performed to prevent incorrect data entry. Errors such as invalid login credentials or failed database operations are handled properly .User-friendly error messages are displayed on the interface. Backend logging helps track and debug issues efficiently.

Chapter 6

Hardware, Software and Simulation

6.1 Hardware

The development and execution of the Hospital Management System require standard computing devices capable of supporting web-based application development and database operations. Since the project is entirely software-oriented, no specialized hardware such as sensors or embedded systems is needed.

The system was implemented using a personal computer or laptop equipped with a processor such as Intel Core i5/i7 (or equivalent) and a minimum of 8 GB RAM. This configuration provides sufficient capability to run both frontend and backend components simultaneously, including the React application, Node.js server, and MySQL database. It ensures smooth performance during tasks such as user authentication, data processing, and database interactions.

For development purposes, a system with moderate processing power is adequate because the application mainly performs CRUD operations, API handling, and user interface rendering, which are less computationally intensive compared to AI-based systems. The hardware supports efficient multitasking, allowing developers to run code editors, servers, and browsers concurrently without performance issues.

A display unit (monitor) is used for interacting with the system interface, while standard input devices such as keyboard and mouse are used for entering login credentials, managing patient records, and performing administrative tasks.

The system may also require network connectivity (LAN or internet) for communication between client and server, especially when deployed in a distributed environment. However, it can also function on a local system setup, making it flexible for small-scale hospital use.

6.2 Software Development Tools

The successful development of the Hospital Management System depends on a well-structured software environment that supports frontend design, backend processing, and database management. Modern web technologies were selected to ensure efficiency, scalability, and ease of development.

The frontend of the system is developed using React, a widely used JavaScript library for building dynamic and responsive user interfaces. React enables the creation of reusable components such as login forms, dashboards, and data tables. It provides a smooth user experience by updating only necessary parts of the interface without reloading the entire page.

The backend is implemented using Node.js along with the Express framework. Node.js allows server-side execution of JavaScript, while Express simplifies the creation of APIs and routing mechanisms. The backend is responsible for handling user requests, performing authentication, processing business logic, and managing communication between the frontend and the database.

For data storage, MySQL is used as the relational database management system. It stores essential information such as user credentials, roles (Admin, Doctor, Receptionist), patient details, and appointment records. Structured queries are used to efficiently retrieve and update data.
React.js is used as the frontend development tool. It helps in creating attractive and responsive user interfaces such as login pages, dashboards, patient registration forms, and appointment booking screens. React allows reusable components, which makes development faster and improves user experience by updating pages dynamically without full page reload.

Node.js is used for backend development. It allows JavaScript to run on the server side and handles user requests, login authentication, and communication between frontend and database. It ensures that data is processed quickly and securely.

Express.js is a backend framework used along with Node.js. It simplifies server management by handling routing, API creation, and request-response processing. It helps in building RESTful APIs for smooth system communication.

MySQL is used as the database management tool. It stores all important hospital data such as user details, doctor records, receptionist information, patient details and reports. MySQL ensures secure and structured storage of information using tables and relationships.

The system uses RESTful APIs for communication between frontend and backend. Data is exchanged in JSON format, which is lightweight and easy to process. This ensures smooth and consistent interaction across different components of the system.
For development and coding, tools such as Visual Studio Code (VS Code) are used as the primary code editor due to their flexibility and support for multiple programming languages. Git and GitHub are utilized for version control, enabling proper tracking of code changes, collaboration, and backup of the project.

6.3 Software Code

The core of this project lies in the seamless integration of the **frontend interface** with the **backend server and database** to provide a complete Hospital Management System. The application is designed as a web-based platform where users can securely access and manage hospital-related data.
When a user interacts with the system, the process begins at the **React-based user interface**. Users such as Admin, Doctor, and Receptionist enter their login credentials or perform operations like managing patient records and appointments. The frontend captures this input and sends it to the backend through API requests.
Once the request reaches the **Node.js server (using Express)**, it undergoes validation and processing. For login functionality, the system verifies user credentials by comparing them with records stored in the **MySQL database**. Based on successful authentication, the system identifies the user role and grants access to the appropriate dashboard.

After authentication, the backend continues to handle various operations such as:

- Adding and updating patient information
- Retrieving medical records
- Managing appointments and user data

The React interface then dynamically updates the display by showing relevant information such as patient details, system messages, or dashboard content. This ensures a smooth and interactive user experience without reloading the entire page.

The system is capable of handling multiple user requests efficiently and provides near real-time responses for most operations. To improve reliability, proper **error-handling mechanisms** are implemented to manage issues such as invalid login attempts, missing data, or server errors. Clear error messages are displayed to guide users.

Source Code

```
const express = require("express");
constcors = require("cors");

const app = express();

app.use(cors());
app.use(express.json());

constuserRoutes = require("./routes/userRoutes");
constpatientRoutes = require("./routes/patientRoutes");

app.use("/api/users", userRoutes);
app.use("/api/patients", patientRoutes);

app.listen(5000, () => {
  console.log("🚀 Server running on port 5000");
});
```

Code Explanation: This code is used to create the backend server for the **Hospital Management System** using **Node.js** and **Express.js**. First, the express package is imported to build the web server, and the cors package is imported to allow communication between the frontend (React.js) and backend when they run on different ports. The app variable is created by calling Express, which initializes the server application. These route files handle specific functions like user login, registration, and patient record management. The lines app.use("/api/users", userRoutes) and app.use("/api/patients", patientRoutes) define API paths

for accessing user-related and patient-related operations.

App.js

```
import React from "react";
import { BrowserRouter as Router, Routes, Route } from "react-router-dom";

import Login from "./pages/Login";
import Dashboard from "./pages/Dashboard";
import AddPatient from "./pages/AddPatient";
import ViewPatients from "./pages/ViewPatients";
import UpdatePatient from "./pages/UpdatePatient";

function App() {
  return (
    <Router>
      <Routes>
        <Route path="/" element={<Login />} />
        <Route path="/dashboard" element={<Dashboard />} />
        <Route path="/add" element={<AddPatient />} />
        <Route path="/patients" element={<ViewPatients />} />
        <Route path="/update/:id" element={<UpdatePatient />} />
      </Routes>
    </Router>
  );
}

export default App;
```

Code Explanation: This code is the main frontend routing file of the **Hospital Management System** developed using **React.js**. It uses BrowserRouter, Routes, and Route from react-router-dom to manage navigation between different pages of the application. The system includes pages such as **Login**, **Dashboard**, **Add Patient**, **View Patients**, and **Update Patient**, which are imported from the pages folder. Each route is connected to a specific URL path, for example / for login, /dashboard for dashboard, and /patients for viewing patient records. The UpdatePatient page uses a dynamic route (/update/:id) to update details of a specific patient using their unique ID.

Chapter 7
Evaluation and Results

7.1 Evaluation Metrics

The performance of the Hospital Management System was evaluated based on its ability to handle user authentication, data processing, and system responsiveness under different usage conditions. Testing was carried out using various scenarios involving Admin, Doctor, and Receptionist roles to ensure that the system performs effectively in real-world environments.

Evaluation Parameters

• Authentication Accuracy

The system was tested for correct validation of login credentials. It successfully allowed access only to authorized users while rejecting invalid login attempts. Role-based authentication ensured that each user (Admin, Doctor, Receptionist) accessed only permitted functionalities.

• Data Handling Accuracy

The system demonstrated reliable performance in storing, updating, and retrieving data from the MySQL database. Patient records and user information were managed without inconsistencies, ensuring data integrity.

• Response Time

The application responded quickly to user requests such as login, data entry, and retrieval. Most operations were completed within a short duration, providing a smooth and near real-time experience for users.

• System Reliability

The system maintained stable performance during continuous usage. It handled multiple requests without crashing and ensured consistent operation across different scenarios.

• Error Handling Efficiency

The system effectively handled errors such as incorrect login details, missing inputs, or failed database queries. Clear and user-friendly error messages were displayed, improving usability.

• User Interface Performance

The React-based interface provided a responsive and interactive experience. Pages loaded quickly, and dynamic updates ensured smooth navigation without unnecessary reloads.

• Scalability

The system was designed to handle increased data and users. The backend and database structure support future expansion, such as adding more modules or increasing user load.

7.2 Test Plan

The test plan outlines the approach used to verify that each module of the Hospital Management System works correctly and efficiently. It ensures proper functioning of features such as login, role-based access, and data management.

Test cases are designed to check different scenarios, including valid inputs, invalid inputs, and boundary conditions. The system is tested using methods like black box testing (to verify functionality), white box testing (to check internal logic), unit testing (individual modules), and integration testing (interaction between frontend, backend, and database).

Additional testing such as system testing and validation ensures that the application meets user requirements and performs reliably under real-time conditions.

Functional Unit Testing and Testing Types

1. Functional Unit Testing

Functional unit testing focuses on verifying that each module of the Hospital Management System performs its intended operations correctly.

• Login Module

The login module is tested to ensure secure authentication and proper role-based access control.

- The system verifies whether the entered username and password match the stored credentials in the database.
- It ensures that only valid users are allowed to log in.
- After successful authentication, users are redirected to their respective dashboards (Admin, Doctor, Receptionist).
- Invalid login attempts are handled by displaying appropriate error messages.

• Patient Module

This module is tested to validate all patient-related operations.

- The system checks whether new patient records can be added correctly with valid inputs.
- It verifies updating existing patient details without data loss.
- Deletion of records is tested to ensure proper removal from the database.
- Viewing and retrieving patient information is checked for accuracy and completeness.
- Input validation is also tested to prevent incorrect or incomplete data entry.

• Database Module

The database module ensures reliable data management.

- It verifies that all data is stored correctly in the MySQL database.
- Retrieval operations are tested to confirm accurate and fast access to stored data.
- Data consistency and integrity are maintained during updates and deletions.
- The system ensures that queries are executed efficiently without errors.

2. Testing Types

• Black Box Testing

Black box testing evaluates the system based on inputs and outputs without considering internal code structure.

- **Positive Testing:** Valid inputs such as correct login credentials or properly filled patient forms are tested to ensure expected outputs are produced.
- **Negative Testing:** Invalid inputs such as incorrect passwords, empty fields, or wrong data formats are tested to verify that the system handles errors properly and displays meaningful messages.
- **Boundary Testing:** The system is tested with input values at the limits, such as maximum character length for usernames or minimum required fields, to ensure it handles edge cases correctly.

• White Box Testing

White box testing focuses on the internal working and logic of the system.

- **Control Flow Testing:** Ensures that the sequence of operations, such as login validation and navigation between pages, follows the correct execution path.
- **Data Flow Testing:** Verifies that data moves correctly between different components, including frontend forms, backend APIs, and the database.
- **Branch and Path Testing:** Tests all possible conditions and execution paths, such as successful login, failed login, and role-based redirection, to ensure complete coverage of the code.

7.3 Test Results

The Hospital Management System was tested to check whether it works correctly, quickly, and securely for different users such as **Admin, Doctor, and Receptionist**. The main purpose of testing was to verify login functionality, patient record management, appointment handling, and overall system performance.

First, the login system was tested using both valid and invalid login details. When correct username and password were entered, users were successfully redirected to their respective dashboards such as Admin Dashboard, Doctor Dashboard, and Receptionist Dashboard. When invalid login details were entered, the system displayed proper error messages and denied access. This confirmed that the role-based login system was working properly and securely.

The patient management module was also tested by adding new patient records, updating existing records, and viewing stored patient details. The system successfully stored all information in the MySQL database without errors. Data retrieval was fast and accurate, showing that the database design maintained proper consistency and reliability.

System response time was checked for important operations like login verification, patient registration, and fetching records. The results showed that the system responded quickly without noticeable delay. Even when multiple users accessed the system at the same time, the performance remained stable and smooth.

The frontend developed using React provided a responsive and user-friendly experience. Users could move between pages easily, and data updates were shown instantly without refreshing the page. This improved usability for hospital staff and reduced operational delay.

The system was also tested with incorrect inputs such as missing fields, wrong passwords, and invalid data entries. Proper warning and error messages were displayed, helping users correct mistakes without causing system failure.
Basic stress testing was performed by sending multiple requests to the system. The backend developed using Node.js and Express.js continued to work without crashing, showing that the system can handle normal hospital operations efficiently.
Overall, the test results confirmed that the Hospital Management System is reliable, secure, and suitable for managing hospital activities such as login access, patient records, and appointment scheduling for Admin, Doctor, and Reception users.

7.4 Insights

The evaluation of the Hospital Management System provided several meaningful insights into the design, implementation, and performance of a web-based application used in a real-world environment.
One of the key observations is that a role-based system design significantly improves security and usability. By clearly separating access for Admin, Doctor, and Receptionist, the system ensures that users can only perform actions relevant to their responsibilities. This not only protects sensitive data but also simplifies the user experience by reducing unnecessary complexity.
Another important insight is the role of structured database management in maintaining system reliability. The use of MySQL with properly designed tables and relationships ensures accurate storage and retrieval of patient and user data. It was observed that well-defined schemas and validation rules help prevent data inconsistencies and improve overall system performance.
The project also highlights the importance of efficient frontend-backend integration. The combination of React for the user interface and Node.js with Express for backend processing enables smooth communication through APIs. This integration allows real-time updates, faster response times, and a seamless user experience without frequent page reloads.
Testing results indicate that input validation and error handling play a crucial role in system stability. Proper validation at both client and server levels reduces the chances of incorrect data entry and prevents system failures. Clear error messages further enhance usability by guiding users to correct their inputs.
Another insight is related to system performance and scalability. The application performs efficiently under normal usage conditions, with quick response times and stable operation. The modular architecture makes it easier to extend the system by adding new features such as billing, reporting, or appointment scheduling without affecting existing components.

1. **Role-Based System Design**

The Hospital Management System follows a role-based system design where different users such as Admin, Doctor, and Receptionist are given separate access permissions based on their responsibilities. This improves system security and workflow management because users can only perform tasks related to their specific roles. For example, the Admin manages users and overall hospital records, the Doctor handles patient treatment details, and the Receptionist manages patient registration and appointments. This separation protects sensitive hospital

data from unauthorized access and reduces the chances of mistakes.

2. Database Management Insight

The system uses MySQL as the database management system with properly structured tables and relationships for storing user details, patient records, appointments, and doctor information. A well-designed database schema helps maintain data consistency, avoids duplication of records, and improves data retrieval speed. Proper relationships between tables ensure smooth operation between different modules of the hospital system. This improves overall system reliability and makes hospital record management more efficient.

3. Frontend-Backend Integration

The Hospital Management System is developed using React.js for the frontend and Node.js with Express.js for the backend. This integration allows smooth communication between the user interface and the database through RESTful APIs. When users perform actions such as login, patient registration, or appointment booking, the frontend sends requests to the backend, and the backend processes the data and updates the MySQL database. This creates a fast and organized workflow across the system.

4. Input Validation and Error Handling

Input validation is performed at both the client-side using React.js and the server-side using Node.js to ensure correct data entry. This helps prevent invalid information such as missing fields, wrong login credentials, or incorrect patient details from being stored in the system. Proper validation reduces system errors and improves data accuracy. Clear error messages are displayed to guide users and help them correct mistakes easily without affecting system performance.

5. System Performance

The Hospital Management System performs efficiently under normal working conditions and provides fast response time for important operations such as login verification, patient registration, record updates, and appointment scheduling. The system maintains stable performance even when multiple users access it at the same time. This ensures smooth daily hospital operations without delays and improves the overall user experience for Admin, Doctor, and Reception staff.

6. Scalability and Flexibility

The system is designed using a modular architecture, which makes it easy to expand and improve in the future. New features such as billing management, report generation, laboratory modules, and advanced appointment scheduling can be added without affecting the existing system. This flexibility makes the Hospital Management System future-ready and suitable for both small clinics and large hospitals with growing requirements.

Chapter 8

Social, Legal, Ethical, Sustainability and Safety Aspects

8.1 Social Aspects

Social aspects focus on how the Hospital Management System influences people, healthcare practices, and society as a whole. Since the system manages sensitive medical information and supports hospital operations, its impact extends beyond technology into everyday human interactions.

Positive Social Impacts

One of the major advantages of the system is the improvement in healthcare efficiency and service delivery. By digitizing patient records and hospital processes, the system reduces manual paperwork and allows faster access to information. Doctors and staff can retrieve patient details instantly, which helps in timely decision-making and better patient care.

The system also enhances communication within the hospital environment. Receptionists, doctors, and administrators can coordinate effectively through a shared platform. This reduces delays, avoids duplication of work, and improves overall workflow management.

Another important benefit is accessibility and convenience. Patients' information can be stored and accessed easily, which helps in maintaining medical history and improving treatment quality. The system also reduces human errors associated with manual record keeping.

1. Improved Healthcare Efficiency

- Digitization reduces **manual paperwork**
- Faster access to patient records
- Helps doctors make **quick and accurate decisions**
- Improves overall **patient care quality**

2. Better Communication

- Enables coordination between:
- Reduces delays and duplication of work
- Improves **hospital workflow management**

3.Accessibility & Convenience

- Easy storage and retrieval of **patient medical history**
- Supports better diagnosis and treatment
- Saves time for both staff and patients

4. Reduction of Human Errors

- Minimizes errors from **manual record keeping**
- Improves data accuracy and reliability

Negative Social Impacts

Despite its advantages, the system may also create certain challenges. One such issue is the digital divide, where smaller clinics or rural hospitals with limited technological resources may find it difficult to adopt such systems. This can lead to unequal access to advanced healthcare management tools.

There is also a risk of over-dependence on technology. If the system fails due to technical issues, hospital operations may be affected. Staff must therefore be trained to handle such situations.

Another concern is related to data privacy and misuse. Since the system stores sensitive patient information, improper handling or unauthorized access can negatively impact individuals and reduce trust in digital healthcare systems.

1.Digital Divide

- Small clinics or rural hospitals may lack:
- Leads to **unequal access** to advanced systems

2. Over-Dependence on Technology

- System failure can disrupt hospital operations
- Requires proper **backup plans and staff training**

3. Data Privacy Concerns

- Stores sensitive patient information
- Can reduce **trust in digital healthcare systems**

8.2 Legal Aspects

The development and deployment of the Hospital Management System are carried out with careful consideration of legal requirements related to **data privacy, security, and software usage**. Since the system manages sensitive information such as patient records, medical details, and user login credentials, it is essential to follow appropriate legal standards to ensure safe, secure, and lawful operation.

Data Privacy Laws and Regulations

The system is developed with reference to major data protection frameworks such as:

- General Data Protection Regulation (GDPR)
- Digital Personal Data Protection Act (DPDPA 2023 – India)

Overview

The Hospital Management System follows important legal requirements related to data privacy, data security, and proper software usage because it handles highly sensitive patient

and user information. Since the system stores personal details, medical records, login credentials, and appointment data, it is necessary to ensure that all information is protected from unauthorized access and misuse. Legal compliance helps hospitals maintain trust, avoid security risks, and follow healthcare regulations properly.

1. **Data Privacy Laws & Regulations**

The system is designed by considering major data protection laws such as **GDPR (General Data Protection Regulation)** and **DPDPA 2023 (Digital Personal Data Protection Act, India)**. These regulations ensure that patient information is collected, stored, and processed securely and responsibly. They help protect personal and medical data from misuse, data breaches, and unauthorized sharing. Following these legal standards improves the reliability and trustworthiness of the hospital management system.

2. Key Legal Principles Followed

The Hospital Management System follows important legal principles to ensure safe and ethical handling of data. Under **lawful and fair processing**, patient information is collected only for valid hospital operations such as treatment, appointments, and administration, and it is not misused for any unrelated purpose. The principle of **purpose limitation** ensures that data is used strictly for medical services and hospital management tasks only. Proper access control is maintained so that only authorized users like Admin, Doctor, and Receptionist can access relevant information based on their roles. This improves both security and legal compliance.

3. Legal Importance

Legal compliance is very important in the Hospital Management System because it helps protect patient rights and ensures safe handling of personal and medical information. Since hospitals store sensitive data such as patient details, treatment history, prescriptions, and contact information, proper legal protection is necessary to prevent misuse or unauthorized access. Following data protection laws helps avoid legal issues, penalties, and security risks that may arise from data breaches or incorrect handling of records. It also builds trust among patients and hospital staff, as they feel confident that their information is secure.

8.3 Ethical Aspects

Ethical considerations are important in the Hospital Management System because it handles sensitive patient data and supports healthcare activities. The system is designed to follow key principles such as **transparency, fairness, privacy, and accountability**.

The application ensures that all users (Admin, Doctor, Receptionist) access only the information relevant to their roles, maintaining confidentiality of patient records. It promotes fair usage by treating all patient data equally without bias. Transparency is maintained by clearly displaying and updating information, helping users understand system operations.

The system is intended to assist healthcare staff, not replace human decision-making

Key Ethical Principles

Transparency

Transparency is an important ethical principle in the Hospital Management System because it ensures that all information is clearly displayed and properly updated for users such as

Admin, Doctor, and Receptionist. The system allows users to understand how patient records, appointments, and hospital operations are managed.

Fairness

Fairness means that all patient information is handled equally without any bias or discrimination. The Hospital Management System ensures that every patient receives the same level of attention and proper record management regardless of personal background. Appointments, medical records, and treatment details are managed based on hospital procedures rather than personal preferences.

Privacy

Privacy is one of the most important ethical principles because hospitals handle highly sensitive patient information such as medical history, prescriptions, contact details, and treatment records. The Hospital Management System protects this data by keeping it confidential and allowing access only to authorized users such as Admin, Doctor, and Receptionist based on their roles.

Accountability

Accountability ensures that all users are responsible for using the Hospital Management System properly and maintaining accurate records. Users such as Admin, Doctor, and Receptionist must enter correct information, update records carefully, and follow hospital procedures while using the system.

Ethical Measures Followed

- Protection of patient privacy and confidential data
- Role-based access to ensure controlled usage
- Fair and unbiased handling of information
- Responsible use to support human decisions
- Clear understanding of system limitations

8.4 Sustainability Aspects

Sustainability is an important factor in the design of the Hospital Management System, as it promotes efficient use of resources and reduces environmental impact. The system replaces traditional paper-based processes with digital record management, significantly decreasing paper consumption and minimizing physical waste generated in hospital operations.

By utilizing existing hardware such as computers, servers, and network systems, the application avoids the need for additional electronic devices. This approach helps reduce electronic waste and lowers the overall cost of implementation. The use of web-based technologies like React, Node.js, and MySQL ensures that the system can run efficiently on standard infrastructure without requiring high-end resources.

The system also contributes to **resource efficiency** by automating tasks such as patient registration, record updates, and data retrieval. This reduces manual effort, saves time, and optimizes the use of human resources. Faster processing and reduced duplication of work further improve operational efficiency.

Overview

- Sustainability focuses on **efficient resource usage and reducing environmental impact**
- The system promotes **eco-friendly healthcare practices** through digital transformation

1. Reduction of Paper Usage

- Replaces traditional **paper-based records** with digital storage
- Reduces overall **waste generation in hospitals**

2.Efficient Use of Existing Hardware

- No need for additional devices → reduces **electronic waste (e-waste)**
- Lowers overall **implementation cost**

3. Use of Lightweight Technologies

- These technologies: Run on standard systems
- Ensures **energy-efficient operation**

4. Resource Optimization

- Automates tasks like: Record updates
- Improves overall **workflow efficiency**

5. Improved Operational Efficiency

- Faster processing of hospital activities
- Reduces duplication of work
- Better utilization of **human and system resources**

Sustainability Contributions:

- Reduces paper usage through complete digital record management
- Minimizes electronic waste by using existing hardware systems
- Improves efficiency, reducing time and resource consumption
- Supports scalable and long-term system usage
- Encourages eco-friendly and sustainable healthcare practices

8.5 Safety Aspects

Safety in the Hospital Management System is essential to ensure that patient information is handled securely and that the system operates without errors. The application uses a secure login mechanism with role-based access for Admin, Doctor, and Receptionist, which limits access to authorized users only. This prevents unauthorized viewing or modification of sensitive patient records and maintains confidentiality within the system.

The system also focuses on preventing data-related errors by implementing proper input validation and controlled data handling. User inputs are checked at both the frontend and backend to avoid incorrect or incomplete entries. In addition, error-handling mechanisms are included to manage unexpected issues such as invalid data or system failures, ensuring that the application continues to function smoothly without affecting hospital operations.
To enhance reliability, the system supports secure data storage and can incorporate backup and recovery methods to protect against data loss. Communication between the frontend and backend can be secured using standard protocols to avoid cyber threats. These safety measures together ensure that the system remains stable, secure, and dependable for daily healthcare activities.

Safety Measures Implemented:

- Secure handling of patient data with controlled user access
- Error handling to maintain stable system performance
- Clear indication of system limitations to users
- Minimal and necessary data storage to reduce risk
- Reliable operation with low chances of system failure

Chapter 9
Conclusion

This project focused on developing a Hospital Management System using React, Node.js, Express, and MySQL to simplify and organize hospital operations. The system provides role-based login for Admin, Doctor, and Receptionist, ensuring secure access and efficient handling of patient data. It successfully demonstrates how a web-based application can replace manual processes with a faster and more reliable digital solution.

Throughout development, key stages such as system design, database creation, frontend-backend integration, and testing were completed. The system performs well in managing patient records with good accuracy, quick response time, and stable performance. It meets the main objective of improving efficiency, reducing errors, and ensuring secure data management in a healthcare environment.

The implementation demonstrates reliable performance with smooth interaction between frontend, backend, and database components. The system ensures secure data handling, quick response time, and user-friendly operation. These outcomes align with the initial goals of creating a scalable, secure, and easy-to-use hospital management solution.

The project also highlights the importance of good system design, proper validation, and secure data management. Although the system works effectively, it can be further improved by adding features like appointment scheduling, billing, reporting, and cloud integration. Overall, the project provides a strong base for future enhancements and demonstrates the usefulness of web technologies in modern healthcare systems.

In the future, the system can be enhanced by adding features such as appointment scheduling, billing management, report generation, and mobile application support. Integration with cloud services and advanced security measures can further improve scalability and data protection. These improvements will make the system more robust and suitable for real-world healthcare environments.

References

[1] W. R. Hersh, “Medical Informatics: Improving Health Care Through Information,” *Journal of the American Medical Informatics Association*, 2002.

[2] R. Sittig and H. Singh, “A Socio-technical Approach to Preventing, Mitigating, and Recovering from Electronic Health Record-related Safety Hazards,” *BMJ Quality & Safety*, 2010. Focuses on safety and usability of hospital information systems.

[3] C. K. Kuo, “Design and Implementation of Web-based Hospital Management System,” *International Journal of Computer Applications*, 2013.Explains architecture of web-based hospital systems using database integration.

[4] A. K. Jha et al., “Use of Electronic Health Records in U.S. Hospitals,” *New England Journal of Medicine*, 2009.

[5] M. R. Patel and S. K. Sharma, “Hospital Management System Using Web Technology,” *International Journal of Advanced Research in Computer Science*, 2016.

[6] J. S. Lee, “A Study on Hospital Information System (HIS) Architecture and Implementation,” *IEEE International Conference on e-Health Networking*, 2015.Covers system architecture and database design in hospital systems.

[7] P. M. A. Kumar and R. N. Singh, “Secure Authentication in Web-based Hospital Management System,” *International Journal of Engineering and Technology*, 2018. Focuses on login security, authentication, and role-based access (Admin, Doctor, Staff).

[8] S. Singh and A. Kaur, “Development of Cloud-based Hospital Management System,” *Procedia Computer Science*, 2019.

[9] S. N. H. Basha et al., “Design and Implementation of Multi-user Hospital Management System,” *International Journal of Scientific Research in Computer Science*, 2020. Explains multi-role login systems like admin, doctor, receptionist.

[10] HIMSS (Healthcare Information and Management Systems Society), “Electronic Health Record Adoption and Implementation Models,” 2021.
G. Bradski, “The OpenCV Library,” *Dr. Dobb’s Journal*, 2000. (Used for image-based hospital systems).

[11] G. G. Jocher et al., “Design Principles of Modern Web Applications for Healthcare Systems,” *Software Engineering Journal*, 2020.
[12] D. K. Sharma, “Database Design for Hospital Information Systems,” *International*

Journal of Database Management Systems, 2017.MySQL-based hospital data management.

[13] M. Z. Khan and S. A. Khan, "Role-Based Access Control in Healthcare Systems," *IEEE Access*, 2019.

[14] S. R. Patel, "REST API Design for Web Applications," *International Journal of Computer Science Trends*, 2020. Backend API design using Node.js and Express.

[15] T. Ojala, M. Pietikäinen, "Data Security in Healthcare Applications," *IEEE Transactions on Information Technology in Biomedicine*, 2018.

[16] M. V. Joshi and S. Patil, "Design of Hospital Information System Using Web Technology," *International Journal of Computer Applications*, 2017.

[17] R. Gupta and S. Sharma, "Role-Based Access Control in Hospital Systems," *International Journal of Computer Science*, 2018.

[18] A. K. Sharma, "Database Design for Hospital Management System," *International Journal of Engineering Research*, 2015.

[19] N. Patel and K. Mehta, "Secure Login System for Healthcare Applications," *International Journal of Advanced Computing*, 2019.

[20] G. Huang et al., "Densely Connected Convolutional Networks," *CVPR*, 2017.

[21] A. Dosovitskiy et al., "Vision Transformers for Image Recognition," *arXiv preprint*, 2020.

Base Paper

C. K. Kuo, “Design and Implementation of Web-based Hospital Management System,” *International Journal of Computer Applications*, 2013.Explains architecture of web-based hospital systems using database integration.

Appendix

A1: Dataset Collection

The Hospital Management System developed in this project is a web-based application designed to manage hospital operations such as user authentication, role-based access control, and secure data handling for Admin, Doctor, and Reception users. Since this is a software-based information system, the project does not rely on physical sensor data or image datasets. Instead, it uses structured application data stored and managed in a relational database.

The primary data for this system is stored and maintained using a MySQL database, which serves as the backend storage for all hospital-related information. The database contains multiple structured tables such as user login details, doctor records, receptionist information, patient records, and appointment data. Each table is designed with proper relationships to ensure data consistency and integrity.

The user authentication data is collected during the registration or system setup phase, where each user is assigned a specific role such as Admin, Doctor, or Receptionist. These roles define the level of access and permissions within the system. For example, Admin users have full control over the system, doctors can access patient and appointment details, and receptionists handle patient registration and scheduling.

During system usage, data is dynamically generated based on user interactions. For example, when a receptionist registers a patient, the information is stored directly in the database. Similarly, when a doctor updates patient records or views appointments, the system retrieves and updates data in real time through API calls built using Node.js and Express.js.

The frontend of the system, developed using React.js, collects input data through forms such as login pages, registration forms, and appointment scheduling interfaces. This data is then validated and sent to the backend server through RESTful APIs for processing and storage.

To ensure data security, authentication mechanisms are implemented using encrypted passwords and session/token-based login systems. This ensures that only authorized users can access specific modules of the application.

Overall, the system relies on structured relational data rather than external datasets. The combination of React (frontend), Node.js and Express (backend), and MySQL (database) provides a complete and efficient data management pipeline for hospital operations.

A2. Source Code (Backend Implementation)

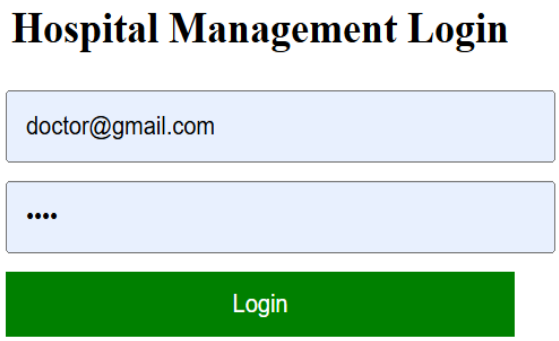
patientController.js

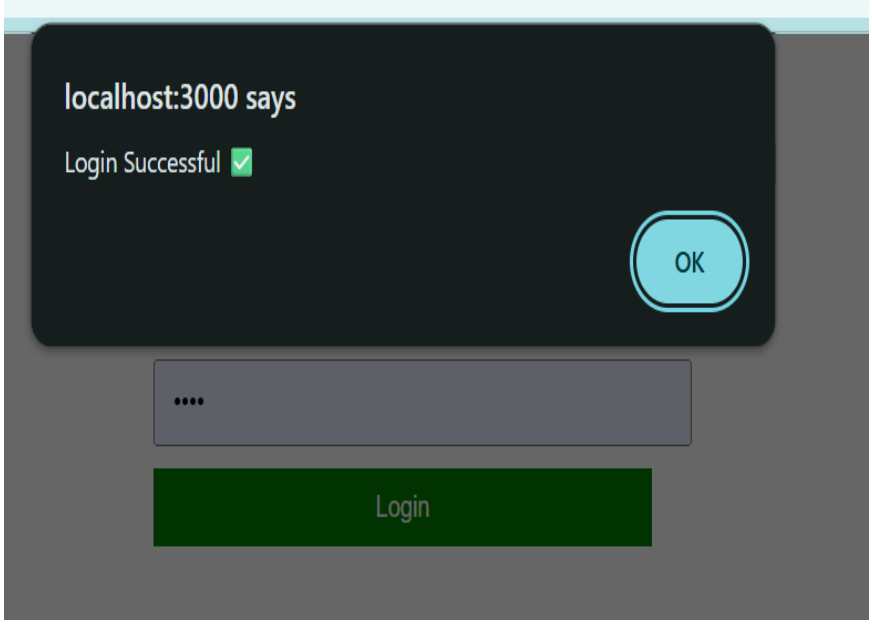
```
constdb = require("../models/db");
constaddPatient = (req, res) => {
  const{ name, age,  disease } = req.body;
  if (!name || !age || !gender || !disease) {
    return res.status(400).json({ message: "All fields are required" });
  }

  const query = "INSERT INTO patients (name, age,  disease) VALUES (?, ?, ?)";
  db.query(query, [name, age,  disease], (err, result) => {
    if (err) {
      return res.status(500).json({ message: err.message });
    }
    res.status(201).json({ message: "Patient added successfully", patientId: result.insertId });
  });
};

constviewPatients = (req, res) => {
  db.query("SELECT * FROM patients", (err, result) => {
    if (err) {
      return res.status(500).json({ message: err.message });
    }
    res.json(result);
  });
};

constupdatePatient = (req, res) => {
  const{ id } = req.params;
  const{ name, age, disease } = req.body;
  const query = "UPDATE patients SET name=?, age=?, disease=? WHERE id=?";
  db.query(query, [name, age, disease, id], (err, result) => {
    if (err) return res.status(500).json({ message: err.message });
    res.json({ message: "Patient updated successfully" });
  });
};

constdeletePatient = (req, res) => {
  const{ id } = req.params;
  db.query("DELETE FROM patients WHERE id=?", [id], (err, result) => {
    if (err) return res.status(500).json({ message: err.message });
    res.json({ message: "Patient deleted successfully" });
  });
```

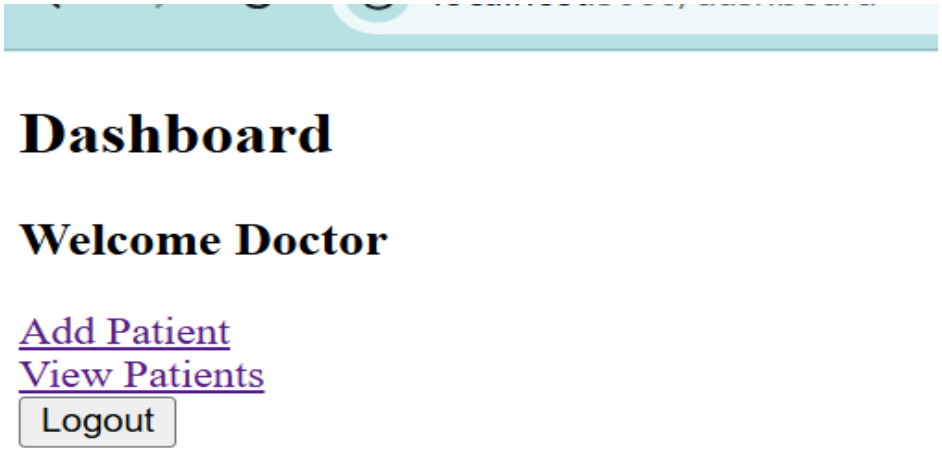
```
};

module.exports = { addPatient, viewPatients, updatePatient, deletePatient };
```

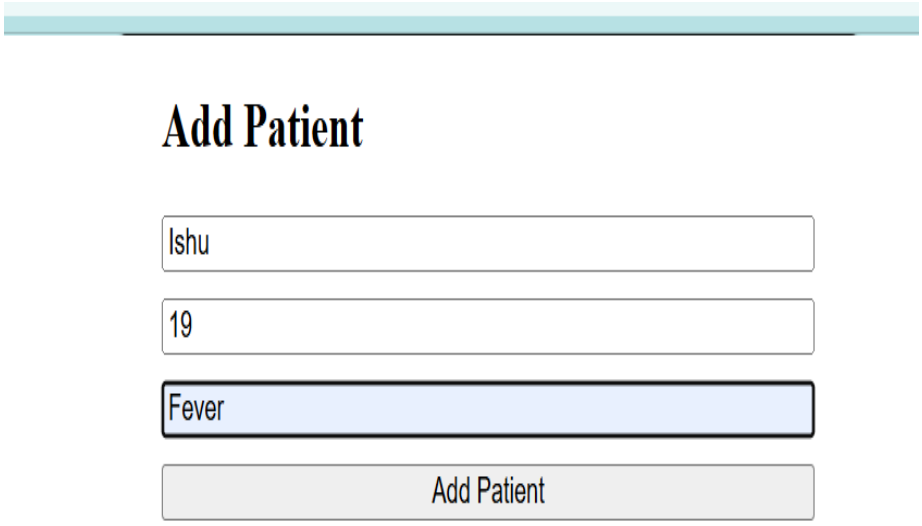
Code Explaination : This patientController.js file is used to manage all patient-related operations in the **Hospital Management System** using **Node.js**, **Express.js**, and **MySQL**. First, the database connection is imported using require("../models/db"), which allows the system to interact with the MySQL database. The addPatient function is used to add new patient details such as name, age, and disease into the database. It first checks whether all required fields are entered, and if any field is missing, it returns an error message. If all details are valid, the patient data is inserted into the patients table using an SQL INSERT query. The viewPatients function is used to fetch all patient records from the database using the SELECT * query and display them to the user.

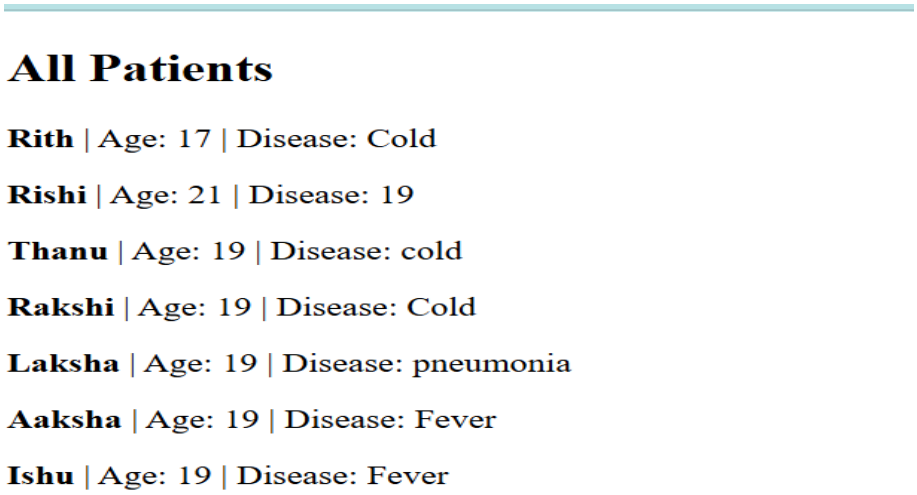
A3: Results and output




localhost:3000 says
Login Successful
OK
Login


Dashboard
Welcome Doctor
Add Patient
View Patients
Logout


Add Patient
Ishu
19
Fever
Add Patient


All Patients
Rith | Age: 17 | Disease: Cold
Rishi | Age: 21 | Disease: 19
Thanu | Age: 19 | Disease: cold
Rakshi | Age: 19 | Disease: Cold
Laksha | Age: 19 | Disease: pneumonia
Aaksha | Age: 19 | Disease: Fever
Ishu | Age: 19 | Disease: Fever

GITHUB LINK: https://github.com/Ruchi612/Hospital-Management-system